

From Java Backend to Flutter Frontend

The WorkBridge Developer's Course — Dart, Flutter, Riverpod & Dio, Taught From Your
Own Repository

A personalized course for MD Mahbubur Rahman Sheikh (MMRSheikh2001)

Built from the `work_bridge_flutter` codebase — Generated August 2026

Contents

1	How To Use This Book	5
2	Orientation: Where You Stand	6
2.1	What You Already Know (from Java / Spring Boot)	6
2.2	What You Need to Learn (the genuine gaps)	6
2.3	Recommended Learning Order	7
3	Technologies Used in the WorkBridge Flutter Project	9
4	Part 1 — Programming Foundations: Dart for a Java Developer	11
4.1	Variables: <code>var</code> , <code>final</code> , <code>const</code>	11
4.2	Core Data Types	11
4.3	Null Safety — the single biggest mental shift from old-style Java	11
4.4	Operators, Control Flow, Loops	12
4.5	Functions: Positional, Named, Optional, Default, Arrow	13
4.6	Classes, Constructors, Named Constructors	13
4.7	Inheritance, Abstract Classes, Mixins	14
4.8	Enums — and why WorkBridge’s are more than they look	14
4.9	Getters, Setters, Static, Generics	15
4.10	Exception Handling	15
4.11	Exercises — Part 1	16
5	Part 2 — Dart Asynchronous Programming	17
5.1	The problem this solves	17
5.2	Synchronous vs Asynchronous	17
5.3	<code>Future</code> , <code>async</code> , <code>await</code>	17
5.4	Try/Catch with <code>async</code> code	18
5.5	<code>Future<T></code> and <code>FutureBuilder</code>	18
5.6	Streams and <code>StreamBuilder</code>	19
5.7	Common Async Mistakes (and how to spot them in your own code)	19
5.8	Connecting This to “How Flutter Waits for a REST API Response”	20
5.9	Exercise Answers: Parts 1–2	20
6	Part 3 — Flutter Fundamentals: The Mental Model	22
6.1	What Flutter actually is	22
6.2	Widget, Widget Tree, Element Tree, Render Tree (beginner level)	22
6.3	<code>StatelessWidget</code> vs <code>StatefulWidget</code>	23
6.4	<code>build()</code> and <code>BuildContext</code> — what they actually mean	24
6.5	<code>setState()</code> — what it actually triggers	24
6.6	Widget Lifecycle	24
6.7	<code>keys</code> — a beginner-level answer to “why do widgets need keys?”	25
6.8	Hot Reload vs Hot Restart	25

6.9 Exercises — Part 3	25
7 Part 4 — Flutter UI and Layout	27
7.1 The constraint model, first — because it explains every “mistake” below	27
7.2 Widget Catalog	27
7.3 Building Screens: the Design → Code Process	31
7.4 Exercise Answers: Part 3	32
8 Part 5 — Building Screens: A Worked Example	33
9 Part 6 — Navigation	34
9.1 Navigator: push, pop, and friends	34
9.2 Named Routes and WorkBridge’s <code>AppRouter</code>	34
9.3 Returning Data From a Screen	35
9.4 Authentication-Based and Role-Based Navigation	35
10 Part 7 — Forms and User Input	37
10.1 <code>TextEditingController</code> and <code>FocusNode</code>	37
10.2 <code>Form</code> , <code>GlobalKey<FormState></code> , and Validation	37
10.3 Disposing Controllers — non-negotiable	38
10.4 Password Fields, Dropdowns, Date Pickers, Image Selection	38
10.5 A Complete Form Example: Registration	38
11 Part 8 — JSON and Models/DTOs	39
11.1 The Chain, In Both Directions	39
11.2 <code>fromJson()</code> in Detail — Using <code>LoginResponse</code> as the Case Study	39
11.3 <code>toJson()</code> in Detail	40
11.4 Nullable Fields — Why APIs Force <code>String?</code> , <code>int?</code> , <code>DateTime?</code>	40
11.5 Nested JSON and Lists	41
12 Part 9 — REST API Fundamentals (From the Client Side)	42
12.1 Status Codes You’ll Actually Handle	43
13 Part 10 — Dio	44
13.1 Why Dio, when <code>http</code> is also a dependency	44
13.2 Setting Up a Dio Instance: <code>BaseOptions</code>	44
13.3 The Five Methods	44
13.4 <code>queryParameters</code> , <code>data</code> , <code>headers</code>	45
13.5 <code>Response</code> and <code>DioException</code>	45
13.6 Interceptors — WorkBridge’s <code>ApiClient</code> , Explained Line by Line	45
13.7 Multipart Requests — Preview	46
14 Part 11 — JWT Authentication: The Complete Flow	47
14.1 The Full Chain	47
14.2 Why <code>flutter_secure_storage</code> and Not <code>shared_preferences</code>	48
14.3 Storing and Restoring the Session	48
14.4 Attaching the Token — the Interceptor, Again	48
14.5 Logout	48
14.6 Expired JWT Handling — the <code>_handleUnauthorized()</code> Flow	49
14.7 Exercises — Part 11	49
15 Part 12 — Riverpod: Taught Properly, Not “It Just Manages State”	51
15.1 The Problem <code>setState</code> Stops Solving	51
15.2 The Core Idea: a Provider is a Recipe, Not a Value	51
15.3 <code>ProviderScope</code> and Where the Graph Actually Lives	52

15.4	The Provider Types WorkBridge Actually Uses	52
15.5	<code>AsyncValue<T></code> — Loading/Success/Error, Unified	53
15.6	<code>ConsumerWidget</code> , <code>ConsumerStatefulWidget</code>	53
15.7	<code>ref.watch</code> vs <code>ref.read</code> vs <code>ref.listen</code> — Why You Can’t Use <code>ref.watch</code> Everywhere . . .	54
15.8	Every Real Provider in WorkBridge, Explained	54
15.9	Exercises — Part 12	55
16	Part 13 — Repository Pattern and Application Architecture	57
16.1	The Layers, and Why Each One Exists	57
16.2	Tracing Three Real WorkBridge Features End to End	58
16.2.1	Feature 1 — Logging In	58
16.2.2	Feature 2 — Searching Jobs	58
16.2.3	Feature 3 — Uploading a Profile Photo	58
16.3	Exercise Answers: Parts 11–13	59
17	Part 14 — File and Image Handling (Your Named Weak Area — Slow, Thorough Treatment)	60
17.1	Selecting an Image: <code>image_picker</code>	60
17.2	Selecting a File: <code>file_picker</code>	60
17.3	Why File Alone Isn’t Enough: the Web/Mobile Split	61
17.4	Uploading a File — the Full Chain	61
17.5	Displaying Images: the Five Cases	62
17.6	URL Construction for Files	62
17.7	Authenticated Network Images — A Real Gap Worth Knowing About	63
17.8	Downloading and Opening Files	63
17.9	File Paths, Bytes, MIME Types, Extensions, Content-Type — Quick Reference	64
17.10	Exercises — Part 14	64
18	Part 15 — Web vs Android vs iOS: Why “localhost” Lies to You	65
18.1	<code>ApiConstants.host</code> , Explained	65
18.2	CORS on Flutter Web	66
18.3	File Access and Download Differences	66
19	Part 16 — Error Handling and Debugging	67
19.1	A Systematic Process (use this instead of guessing)	67
19.2	Categories of Errors You’ll Actually Hit	67
19.3	Why <code>ListView</code> Sometimes Causes Layout Errors, <code>NoSuchMethodError</code> , and Provider Rebuilds — Answered Together	68
20	Part 17 — Testing (Fundamentals, Kept Practical)	69
20.1	The Three Levels	69
20.2	Mocking	69
20.3	Testing a Provider	70
20.4	Exercise Answers: Parts 14–17	70
21	Part 18 — Build a Mini WorkBridge From Scratch: “TaskBridge”	71
21.1	1–2. Login + JWT Authentication	71
21.2	3–4. Home Screen + API Connection	72
21.3	5–6. List Data + Details Page	72
21.4	7. Search	73
21.5	8. Form Submission	73
21.6	9–10. Image Upload + File Upload	73
21.7	11. Profile	73
21.8	12–13. State Management + Error Handling (tie everything together)	73
21.9	14. Navigation	73

21.1015. Logout	73
22 Part 19 — Reading Your Own Codebase, Guided	75
22.1 ProfileGate — Role-Based Widget Branching	75
22.2 GigRepository — Confirming the Pattern Holds Across Modules	76
22.3 Cross-Module Consistency Check — Do This Yourself	76
23 Part 20 — Designing a New Feature Without AI: “Saved Jobs” Worked Example	77
23.0.1 Question 1: What backend endpoints does this need, and what HTTP verbs?	77
23.0.2 Question 2: What DTOs do you need on the Flutter side?	77
23.0.3 Question 3: What does the repository method set look like?	77
23.0.4 Question 4: What providers do you need, and of which type — walk through it using Part 12’s decision framework?	78
23.0.5 Question 5: What does the UI layer need to handle, following Part 4/12’s discipline? .	78
23.0.6 Question 6: What could go wrong, and how does existing WorkBridge code already show you the fix?	78
24 Part 21 — FAQ / Troubleshooting Guide	80
25 Part 22 — Exercise Index and Self-Assessment	82
25.1 Self-Assessment Checklist	82
26 Appendix A — Project-Specific Concept Index	84
27 Appendix B — The 30 Skills You Now Have	86
28 Closing Note	87

Chapter 1

How To Use This Book

You already know how to make WorkBridge *work*. That’s not the same as understanding *why* it works, and it’s the gap that keeps you dependent on AI to write new features. This book closes that gap.

The rule this whole book follows: nothing is explained in the abstract without being connected back to a real file in your `work_bridge_flutter` repository. When you see a concept, you will also see exactly where it lives in your code, so the concept stops being “a Flutter thing” and becomes “the thing that makes `AuthController` work.”

How the chapters are built. Most chapters follow the same rhythm:

1. **The problem** — what goes wrong without this concept.
2. **The concept**, explained from first principles, with a Java/Spring comparison where one genuinely applies (never a forced one).
3. **A minimal example** you could type yourself in two minutes.
4. **The WorkBridge example** — the real code, explained line by line.
5. **Exercises** at the end, in three tiers: Recall, Implementation, Problem-Solving. Answers are collected at the end of the relevant part so you’re not tempted to peek early.

What “understanding” means for this book. By the end, you should be able to open a blank Flutter project and, without generating code from an AI, build a screen that logs a user in against a Spring Boot backend, lists data from a paginated search endpoint, lets the user upload a photo, and handles the loading/error/success states correctly. Part 18 makes you actually do this, from scratch, before you return to your real project in Part 19.

A note on honesty. Where WorkBridge does something unusual — an inconsistent naming pattern, a manual DTO instead of generated code, a `late` field that could be `final` — this book says so, and explains the tradeoff, instead of pretending it’s the only correct way. You built this without following a textbook, and some of it reflects that. That’s not a criticism; understanding *why* an implementation is unusual is more valuable to you right now than a clean textbook example would be.

Chapter 2

Orientation: Where You Stand

2.1 What You Already Know (from Java / Spring Boot)

You built and secured a 41-controller Spring Boot backend, including finding and fixing `@PreAuthorize` gaps and IDOR issues. That means you already understand these ideas — you just don't yet have the Dart/Flutter *vocabulary* for them:

You already understand (Java/Spring)	Its Dart/Flutter equivalent
Classes, constructors, interfaces, inheritance	Dart classes, constructors, abstract classes — syntax differs, concept doesn't
DTOs (Request/Response objects)	Dart DTOs with <code>fromJson/toJson</code> — same <i>purpose</i> , hand-written instead of Lombok-generated
A layered architecture (Controller → Service → Repository)	UI → Provider/Controller → Repository → ApiClient
Dependency injection via Spring's <code>ApplicationContext</code>	Dependency injection via Riverpod's <code>ProviderScope</code>
<code>@RestController</code> methods returning JSON	Dio calls that receive JSON and parse it into a DTO
JWT issued by your <code>AuthController</code> , validated by a filter	The <i>other end</i> of the same JWT — attaching it, storing it, reacting to expiry
<code>CompletableFuture</code> / async service calls	Dart's <code>Future</code> and <code>async/await</code>
Bean validation (<code>@NotNull</code> , <code>@Email</code>)	<code>TextFormField</code> validators
<code>MultipartFile</code> on the Spring side	<code>MultipartFile</code> on the Dio side (yes, same name, same idea, different library)

You are not starting from zero. You are starting from “I know what this is supposed to do on the server; I don't yet know how to make the client do its half.”

2.2 What You Need to Learn (the genuine gaps)

Based on WorkBridge's actual code, your real gaps are concentrated in a few places, roughly in order of how much they'll block you:

1. **The Flutter widget/rebuild mental model** — this has no Spring Boot analogue at all. A `build()` method is not a “controller method that runs once”; it's a *rendering function that Flutter calls over and over*. Get this wrong and everything above it feels magical.

2. **Riverpod** — WorkBridge uses `StateNotifierProvider`, `FutureProvider`, `StateProvider`, and `Provider` together. You need to understand *why there are four kinds*, not just copy whichever one is nearby.
3. **The Dio → interceptor → JWT chain** — you understand JWTs from the server side; you haven't yet had to attach one automatically to every outgoing request and react when it expires.
4. **Multipart file/image upload** — genuinely one of the more fiddly parts of any mobile client, and it's the part you named as your weakest area. Part 14 is dedicated entirely to this.
5. **Manual JSON serialization discipline** — no Lombok, no Jackson. Every `fromJson/toJson` in WorkBridge is hand-written, so a typo in a key name fails silently at runtime instead of at compile time. You need the debugging instinct for this specifically.

2.3 Recommended Learning Order

```
Dart basics (Part 1)
↓
Dart OOP (Part 1)
↓
Future / async-await (Part 2)
↓
Flutter widgets & the widget tree (Part 3)
↓
State & setState (Part 3)
↓
Layout widgets (Part 4)
↓
Building a screen end-to-end (Part 5)
↓
Navigation (Part 6)
↓
Forms (Part 7)
↓
JSON / DTOs (Part 8)
↓
REST API concepts (Part 9)
↓
Dio (Part 10)
↓
JWT auth flow (Part 11)
↓
Riverpod (Part 12)
↓
Repository/architecture pattern (Part 13)
↓
File & image handling (Part 14)
↓
Platform differences, debugging, testing (Parts 15-17)
↓
Build a mini app from scratch (Part 18)
↓
Walk through the real WorkBridge code (Part 19)
↓
Design new features without AI (Part 20)
```


Each link in this chain depends on the one above it — you cannot really understand Riverpod's **FutureProvider** without understanding **Future** first, and you cannot understand *why* WorkBridge needs a repository layer without having felt the pain of putting Dio calls directly in a widget at least once. Don't skip ahead.

Chapter 3

Technologies Used in the WorkBridge Flutter Project

This list comes directly from your `pubspec.yaml` — nothing here is included because it's popular; it's included because your project actually depends on it.

Package	What it is	Problem it solves in WorkBridge
<code>flutter_riverpod</code> (v3)	State management + dependency injection library	Owns auth state, search filters, and every screen's loading/error/data state
<code>dio</code>	HTTP client	Every network call to your Spring Boot API goes through it
<code>flutter_secure_storage</code>	Encrypted key-value storage	Stores the JWT and cached user object so they survive app restarts
<code>jwt_decoder</code>	Reads claims out of a JWT string	Available for reading token expiry (used less than storage — see Part 11)
<code>shared_preferences</code>	Plain (non-encrypted) key-value storage	Present as a dependency; secure storage is used for anything sensitive
<code>provider</code>	An older, simpler DI/state library	Present alongside Riverpod (a legacy dependency you should be aware you no longer need for new code)
<code>image_picker</code>	Opens the OS gallery/camera and returns a picked image	Profile photo upload, portfolio images
<code>file_picker</code>	Opens the OS file browser and returns a picked file	Resume/CV upload, training certificates
<code>url_launcher</code>	Opens a URL in the browser/external app	Opening uploaded files/links outside the app
<code>http</code>	A lower-level HTTP client than Dio	Present as a dependency but Dio is what you actually call — see Part 10 for why a project settles on one

Every one of these gets its own deep dive later — this table is just your map. The two you'll spend the

most time with are **Dio** (Part 10) and **Riverpod** (Part 12), because together they define almost everything about how a WorkBridge screen is structured.

Chapter 4

Part 1 — Programming Foundations: Dart for a Java Developer

Dart is a class-based, statically-typed, null-safe language — in that sense it feels closer to Java than to JavaScript, even though it compiles to run inside Flutter the way JS runs in a browser. Where the two diverge, this chapter says so explicitly.

4.1 Variables: `var`, `final`, `const`

```
var name = 'Mahbub';           // type inferred as String, can be reassigned
final city = 'Dhaka';          // type inferred, CANNOT be reassigned after this line
const pi = 3.14159;            // like final, but its value must be known at COMPILE time
```

Java comparison: `final` in Dart `final` in Java — assign once. `const` is stricter: it must be computable before the program even runs, so `const now = DateTime.now();` is illegal (the time isn't known at compile time), but `final now = DateTime.now();` is fine. You'll see `const` constantly in Flutter widget code (`const SizedBox(height: 16)`) — it's a performance hint telling Flutter “this widget never changes, don't even consider rebuilding it.”

In WorkBridge: `lib/services/storage_service.dart` declares `static const token = 'wb_token';` — a compile-time constant string key, exactly like a Java `public static final String TOKEN_KEY`.

4.2 Core Data Types

`String`, `int`, `double`, `bool`, `List<T>`, `Set<T>`, `Map<K, V>` — conceptually identical to Java's `String`, `int`, `double`, `boolean`, `List<T>`, `Set<T>`, `Map<K,V>`. The differences that matter:

- Dart has no primitive/boxed distinction — `int` is always an object, always nullable-capable.
- Dart's `List` literal syntax is lighter: `final skills = ['Java', 'Dart', 'SQL'];` instead of `List.of(...)`.
- Map literals: `final json = {'id': 1, 'name': 'John'};` — this exact shape is what a decoded JSON response looks like, and it's central to Part 8.

4.3 Null Safety — the single biggest mental shift from old-style Java

Dart is *sound null-safe*: every type is non-nullable unless you explicitly mark it nullable with `?`.

```
String name = 'Mahbub';    // can NEVER be null - the compiler guarantees it
String? nickname;          // CAN be null; starts as null
```

If `nickname` is `String?`, the compiler will not let you call `nickname.length` directly — you must prove it isn't null first:

```
if (nickname != null) {
  print(nickname.length);    // safe: Dart "promotes" nickname to String here
}

print(nickname?.length);     // null-aware call: returns null if nickname is null
print(nickname!.length);     // "trust me" operator: crashes at runtime if null
print(nickname ?? 'no nickname'); // fallback value if null
```

Why this matters for WorkBridge specifically: every field coming from your Spring Boot JSON responses is a *guess* about what the server sends. Look at `auth/response/login_response.dart`:

```
class LoginResponse {
  final String? token;
  final String tokenType;
  final int? userId;
  ...
  factory LoginResponse.fromJson(Map<String, dynamic> json) {
    return LoginResponse(
      token: json['token'] as String?,
      tokenType: json['tokenType'] as String? ?? 'Bearer',
      userId: (json['userId'] as num?)?.toInt(),
      ...
    );
  }
}
```

Almost every field is `String?` or `int?` because the Dart side cannot *guarantee* the JSON contains that key — a Spring Boot response is just a `Map<String, dynamic>` as far as Dart is concerned, and Dart's compiler has no idea what your `LoginResponseDTO.java` actually looks like. `tokenType` is the one exception — it's non-nullable because the code supplies a fallback (`?? 'Bearer'`) at the exact moment it's parsed, guaranteeing it's never actually null once the object exists. This is the pattern you should copy: nullable while parsing, with defaults resolved as early as possible so the rest of your app doesn't have to keep checking for null.

`(json['userId'] as num?)?.toInt()` looks strange the first time you see it — it exists because JSON numbers decode into Dart's `num` type, which could be either `int` or `double` depending on how the server serialized it, so you defensively cast to `num?` first and then convert.

4.4 Operators, Control Flow, Loops

These are close enough to Java that a table is more useful than prose:

Concept	Dart	Java	Difference
If/else	<pre>if (x > 0) {...} else {...}</pre>	same	none
Switch	<pre>switch (status) { case Active: ...; }</pre>	same shape	Dart's newer switch <i>expressions</i> (<code>switch (x) { Active => ..., }</code>) have no Java equivalent pre-21
For loop	<pre>for (var i = 0; i < 5; i++) {}</pre>	same	none

Concept	Dart	Java	Difference
For-in	for (final job in jobs) {}	for (Job job : jobs)	same idea, different keyword
While	while (cond) {}	same	none
Ternary	x > 0 ? 'positive' : 'negative'	same	none
Null-coalescing assign	value ??= 'default';	no direct equivalent	assigns only if value is currently null
Spread in collection	[...listA, ...listB]	no direct equivalent	expands a list inline into another literal
Collection-if	[if (isAdmin) 'settings']	no equivalent	conditionally includes an item in a list literal

You'll see collection-if and spread constantly in Flutter's children: [...] lists — e.g. `login_screen.dart`'s `if (_errorMessage != null) ErrorBanner(message: _errorMessage!)` inside a `Column`'s children list is a *collection-if*: it conditionally inserts a whole widget into the widget tree, which is Flutter's version of a JSX conditional render.

4.5 Functions: Positional, Named, Optional, Default, Arrow

```
// Positional required
int add(int a, int b) => a + b;           // arrow function = single-expression shorthand for { return a + b; }

// Positional optional (square brackets)
String greet(String name, [String greeting = 'Hello']) => '$greeting, $name';

// Named parameters (curly braces) - THIS is the one you'll use constantly in Flutter
Widget buildCard({required String title, String? subtitle, int elevation = 1}) {
  return Card(elevation: elevation.toDouble());
}
```

Named parameters are why Flutter widget constructors read like configuration: `TextFormField(controller: _emailCtrl, keyboardType: TextInputType.emailAddress, decoration: ...)`. Every one of those is a named argument, most marked required in the widget's constructor, and Dart's tooling will refuse to compile if you forget a required one — this is your closest equivalent to Spring's `@RequiredArgsConstructor` validation, except caught by the compiler instead of at bean-creation time.

4.6 Classes, Constructors, Named Constructors

```
class UserProfile {
  final String name;
  final int? age;

  UserProfile(this.name, this.age);           // default constructor, shorthand for
                                           // UserProfile(String name, int? age) { this.name = name; this.age = age; }

  UserProfile.guest() : name = 'Guest', age = null; // NAMED constructor - an alternate factory-like constructor
}
```

`this.name` directly in the parameter list is Dart's shortcut for “take this constructor argument and assign it straight to the field of the same name” — there's no equivalent shorthand in Java; you'd write the assignment

out by hand in the constructor body.

In **WorkBridge**, `LoginResponse` uses this pattern combined with a **factory** constructor:

```
factory LoginResponse.fromJson(Map<String, dynamic> json) { ... }
```

factory tells Dart “this named constructor doesn’t necessarily create a brand-new instance the normal way — it runs arbitrary logic first and returns *an* instance.” That’s exactly what parsing JSON needs: you have to read the map, decide what to do with missing keys, and only then build the object.

4.7 Inheritance, Abstract Classes, Mixins

```
abstract class Shape {
  double area();           // no body - subclasses MUST implement this
}

class Circle extends Shape {
  final double radius;
  Circle(this.radius);
  @override
  double area() => 3.14159 * radius * radius;
}
```

This is identical in spirit to a Java **abstract class**. Dart doesn’t have a separate **interface** keyword — *any* class can be used as an interface via **implements**, and **abstract class** is used when you want to share some real code too, exactly like Java’s abstract classes versus its interfaces (pre-default-methods).

Mixins (`mixin`, `with`) have no clean Java equivalent (they’re closer to default methods on multiple interfaces at once). You’ll see them less in your day-to-day **WorkBridge** code, but Flutter’s own framework leans on them heavily (`SingleTickerProviderStateMixin` for animations, for example).

You won’t find much inheritance in **WorkBridge**’s DTOs — each is a flat, independent class. That’s a deliberate (if implicit) choice: it keeps `fromJson/toJson` simple at the cost of some repetition between similar DTOs. A “cleaner” architecture might introduce a shared base class or use code generation (`json_serializable`) to eliminate the repetition — **WorkBridge** doesn’t, which is a completely reasonable tradeoff for a project at this size, but worth knowing it’s a choice, not a law.

4.8 Enums — and why **WorkBridge**’s are more than they look

```
enum UserRole { USER, COMPANY, ADMIN }
```

That’s a plain enum. But **WorkBridge**’s `auth/request/user_request.dart` extends it:

```
enum UserRole {
  USER,
  COMPANY,
  ADMIN;

  String toJson() => name.toUpperCase();

  static UserRole? fromJson(String? role) {
    if (role == null) return null;
    return UserRole.values.firstWhere(
      (e) => e.name.toLowerCase() == role.toLowerCase(),
      orElse: () => UserRole.USER,
    );
  }
}
```

```

    );
  }
}

```

Dart enums can have methods, exactly like a Java enum can. The semicolon after `ADMIN` is required *only* because members follow — it separates “the list of enum values” from “the class body.” `toJson()/fromJson()` here mirror the same pattern every DTO uses, applied to an enum: your Spring Boot backend sends role as a plain string (`"ADMIN"`), and this pair of methods is the translation layer between that string and Dart’s typed `UserRole.ADMIN`. Notice the defensive `orElse: () => UserRole.USER` — if the backend ever sent an unrecognized role string, this silently defaults to `USER` rather than crashing. That’s a deliberate safety net, though it does mean a real bug in the backend’s role string could go unnoticed in this codepath — worth being aware of as a tradeoff, not treating as a bug in itself.

4.9 Getters, Setters, Static, Generics

```

class ApiConstants {
  ApiConstants._(); // private named constructor - prevents instantiation
  static String get host { ... } // a GETTER: called like a field (ApiConstants.host), runs co
  static String get baseUrl => 'http://$host:8090/api/';
}

```

This is a real pattern lifted straight from `lib/utils/api_constants.dart`. `ApiConstants._()` is a common Dart idiom for “this class is just a namespace for static members, nobody should ever call `ApiConstants()`” — the underscore prefix makes the constructor private to the file, and since it’s the *only* constructor, the class becomes effectively un-instantiable. The Java equivalent is a class with a private constructor and only static members (like `java.lang.Math`).

`static String get host` is a getter — it looks like a field from the outside (`ApiConstants.host`, no parentheses) but actually runs a small decision tree every time it’s accessed (see Part 15 for exactly what that logic does).

Generics (`List<T>`, `Map<K, V>`, `Future<T>`) work exactly like Java generics conceptually — type-safety enforced at compile time, erased-ish at runtime. You’ll write `Future<List<JobResponseDTO>>` constantly, and it means exactly what it looks like: “eventually, a list of job DTOs.”

4.10 Exception Handling

```

try {
  final response = await dio.get('/jobs/15');
} on DioException catch (e) {
  print('Network error: ${e.message}');
} catch (e, stackTrace) {
  print('Unexpected error: $e');
} finally {
  print('Always runs');
}

```

Same shape as Java’s `try/catch/finally`. `on DioException catch (e)` is Dart’s way of catching a *specific* exception type first (like Java’s `catch (DioException e)`), falling through to a generic `catch (e, stackTrace)` for anything else. You’ll see this exact pattern in `login_screen.dart`’s `_login()` method, and it’s the backbone of Part 16 (error handling).

4.11 Exercises — Part 1

Level 1 — Recall 1. What’s the difference between `final` and `const`? 2. Why is `String?` different from `String` at the type level, not just “sometimes null”? 3. What does the `factory` keyword change about a constructor?

Level 2 — Implementation 1. Write a `JobLevel` enum (`ENTRY`, `MID`, `SENIOR`) with a `fromJson/toJson` pair following the `UserRole` pattern above. 2. Write a `Money` class with a named constructor `Money.zero()` that sets an amount of `0.0` and a currency of `'BDT'`. 3. Write a function `String formatSalary({required double amount, String currency = 'BDT'})` using named parameters with a default.

Level 3 — Problem Solving 1. You’re given this JSON from a new endpoint: `{"jobId": 12, "title": "Backend Dev", "salary": null, "tags": ["java","spring"]}`. Sketch the Dart class (fields + types only, no full `fromJson` yet) you’d need to represent it, deciding which fields should be nullable and why.

(Answers for all Part 1–2 exercises are in the “Exercise Answers: Parts 1–2” section at the end of Part 2.)

Chapter 5

Part 2 — Dart Asynchronous Programming

5.1 The problem this solves

Network calls take time. If Dart ran `dio.get('/jobs/15')` the same way it runs `2 + 2`, your entire app — the UI included, since Flutter is single-threaded for UI work — would freeze until the Spring Boot server replied. `Future` and `async/await` exist to let your code say “start this, and come back to the rest of the function *later*, once it’s done, without blocking anything in the meantime.”

5.2 Synchronous vs Asynchronous

```
// Synchronous: each line waits for the previous one, blocking the whole thread
int square(int x) => x * x;
print(square(4));    // prints 16 immediately

// Asynchronous: the function returns a "promise of a future value" immediately,
// and the actual value arrives later
Future<int> fetchScore() async {
  await Future.delayed(const Duration(seconds: 2)); // simulate network delay
  return 42;
}
```

Calling `fetchScore()` does **not** pause your program for two seconds. It immediately returns a `Future<int>` — an object that represents “a 42 that doesn’t exist yet, but will.” Java’s `CompletableFuture<Integer>` is the closest thing you already know: same idea, “a value, eventually,” but Dart bakes the language keywords `async/await` directly around it instead of leaving you to chain `.thenApply()`.

5.3 Future, async, await

```
Future<UserProfileResponseDTO> loadProfile(int id) async {
  final response = await _dio.get(ApiConstants.userProfileById(id));
  return UserProfileResponseDTO.fromJson(response.data);
}
```

Read this line by line:

- `Future<UserProfileResponseDTO>` — the return type. This function doesn't return a `UserProfileResponseDTO` directly; it returns a *promise* of one.
- `async` — marks the function body as one that's allowed to use `await` inside it. Marking a function `async` also automatically wraps whatever it `returns` in a `Future`, even if you write plain `return dto;`.
- `await _dio.get(...)` — “pause *this function* (not the whole app) until the `Future` from `_dio.get` completes, then unwrap its value into `response`.” Everything after this line simply doesn't run until the HTTP call finishes.

What actually happens at runtime: when Dart hits `await`, it suspends *only this function call* and hands control back to the event loop, which is free to keep the UI responsive, run other code, or start other `async` work in the meantime. When the awaited `Future` completes (the HTTP response arrives), Dart resumes this function exactly where it left off. This is why a Flutter app doesn't freeze while a `dio.get` call is in flight — nothing else is blocked, only this one call chain is “paused.”

5.4 Try/Catch with `async` code

```
Future<void> login(String email, String password) async {
  state = const AsyncValue.loading();
  try {
    final repo = _ref.read(authRepositoryProvider);
    final res = await repo.login(LoginRequest(email: email, password: password));
    state = AsyncValue.data(res);
  } catch (e, st) {
    state = AsyncValue.error(e, st);
    rethrow;
  }
}
```

This is your actual `AuthController.login()` from `auth/auth_provider.dart`. The `try/catch` around an `await` works exactly like a synchronous `try/catch` — if the HTTP call throws (bad credentials, network down, server error), execution jumps straight to `catch`, skipping the `state = AsyncValue.data(res)` line entirely. `rethrow` (as opposed to `throw e`) re-throws the *original* exception with its original stack trace intact, which matters for debugging: `throw e` would make the error look like it originated at the `throw` line instead of wherever it actually happened.

5.5 `Future<T>` and `FutureBuilder`

Outside of Riverpod, Flutter's own primitive for “build UI from a `Future`” is `FutureBuilder`:

```
FutureBuilder<JobResponseDTO>(
  future: jobRepository.getJobById(15),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const CircularProgressIndicator();
    }
    if (snapshot.hasError) {
      return Text('Error: ${snapshot.error}');
    }
    return Text(snapshot.data!.title);
  },
)
```

`WorkBridge` doesn't use `FutureBuilder` anywhere — it uses Riverpod's `FutureProvider` + `.when()` instead (Part 12), which solves the same problem but also gives you caching, auto-refetch on dependency

change, and avoids re-triggering the future every time the widget rebuilds (a genuine footgun with raw `FutureBuilder`, since a `future`: re-created inline on every `build()` restarts the request). It's worth knowing `FutureBuilder` exists because you'll see it in tutorials and other people's code, but for WorkBridge-style architecture, `FutureProvider` is the better tool and the one you should keep reaching for.

5.6 Streams and StreamBuilder

A `Future` resolves once. A `Stream` can emit many values over time — think of it as an async version of an `Iterable`, or a `Java Flow.Publisher`.

```
Stream<int> countdown() async* {
  for (var i = 5; i > 0; i--) {
    await Future.delayed(const Duration(seconds: 1));
    yield i;
  }
}
```

`async*` (not just `async`) marks a *generator* function that produces a stream; `yield` emits one item at a time instead of returning once. `StreamBuilder` consumes a stream the way `FutureBuilder` consumes a future, rebuilding every time a new value arrives.

When streams are useful, and where WorkBridge would use one if it needed live updates: a chat feature is the textbook case — new messages arriving over time is exactly a stream's shape. Your `chat/` module currently fetches messages via ordinary `Future`-returning Dio calls (poll-and-refresh), not a live `Stream` — which is a completely valid choice for a project at this stage, but if you ever add WebSocket-based real-time chat, that connection would naturally be modeled as a `Stream<Message>`, and Riverpod's `StreamProvider` would consume it the same way `FutureProvider` consumes a `Future` today.

5.7 Common Async Mistakes (and how to spot them in your own code)

1. **Forgetting `await`.** `final res = repo.login(...)`; without `await` gives you a `Future<LoginResponse>`, not a `LoginResponse` — trying to use `res.token` won't even compile, since `Future` has no `.token` getter. This is usually caught immediately by the type checker, which is a real advantage over Java's checked-but-easy-to-swallow exceptions.
2. **`async` without `await` inside** — legal, but almost always a sign you meant to make the function synchronous.
3. **Not handling the error case.** An awaited call that throws and isn't wrapped in `try/catch` propagates the exception up to whoever called your function — in a Riverpod `FutureProvider`, that's *fine* (the provider catches it and turns it into an `AsyncValue.error` automatically, see Part 12), but in a raw `onPressed`: callback with no `try/catch`, it becomes an unhandled exception that Flutter will report to the console and the UI will silently fail to update.
4. **Racing state after `await`.** Look at this pattern from `login_screen.dart`:

```
await ref.read(authControllerProvider.notifier).login(...);
if (!mounted) return;
Navigator.of(context).pushNamedAndRemoveUntil('/', (route) => false);
```

The `if (!mounted) return;` guard exists because the user could navigate away or the widget could be disposed *while the `await` is in flight* — by the time it resumes, `context` might belong to a widget that's no longer in the tree. Using `context` after an `await` without checking `mounted` first is one of the most common real Flutter bugs, and WorkBridge is careful about it here.

5.8 Connecting This to “How Flutter Waits for a REST API Response”

Trace the full chain for a single button press, so the `await` above stops being abstract:

```
User taps "Sign In"
↓
_login() (async) starts running
↓
await ref.read(authControllerProvider.notifier).login(email, password)
  ↓ (AuthController.login, also async)
await repo.login(LoginRequest(...))
  ↓ (AuthRepository.login, also async)
await _dio.post(ApiConstants.login, data: dto.toJson())
↓
Dio hands the HTTP request to the OS network stack
  ↓ <-- this is the only real "wait": milliseconds to seconds,
  ↓      during which the UI thread is FREE, not blocked
Spring Boot receives it, authenticates, returns JSON + JWT
↓
Dio receives the response, resumes AuthRepository.login()
↓
LoginResponse.fromJson(...) parses it
↓
storageService.saveSession(loginRes) persists the token
↓
AuthRepository.login() returns, resuming AuthController.login()
↓
state = AsyncValue.data(res) → every widget watching this provider rebuilds
↓
AuthController.login() returns, resuming _login() in the widget
↓
Navigator pushes the home route
```

Every `↓` in this chain is an `await` boundary. Nothing above it blocks the UI thread; each function simply pauses until the one below it finishes, and Dart’s event loop is free to paint frames, respond to taps, and run other code the entire time.

5.9 Exercise Answers: Parts 1–2

Part 1, Level 1

1. **final**: assign once, but the value can be computed at runtime (e.g. `DateTime.now()`). **const**: assign once *and* the value must be known at compile time.
2. **String** can never hold `null` — the compiler enforces it and will not compile code that assigns `null` to it. **String?** is a genuinely different type that *includes* `null` as a valid value, and the compiler forces you to check before using members on it.
3. **factory** lets a constructor run arbitrary logic and decide what to return (including, in some designs, returning a cached/existing instance) instead of unconditionally allocating a new object field-by-field.

Part 1, Level 2 — sample solutions:

```
enum JobLevel {
  ENTRY, MID, SENIOR;
  String toJson() => name.toUpperCase();
  static JobLevel? fromJson(String? v) {
    if (v == null) return null;
    return JobLevel.values.firstWhere(
```

```

        (e) => e.name.toLowerCase() == v.toLowerCase(),
        orElse: () => JobLevel.ENTRY,
    );
}
}

class Money {
    final double amount;
    final String currency;
    Money(this.amount, this.currency);
    Money.zero() : amount = 0.0, currency = 'BDT';
}

String formatSalary({required double amount, String currency = 'BDT'}) =>
    '$currency ${amount.toStringAsFixed(2)}';

```

Part 1, Level 3 — reasonable fields: `int jobId` (non-nullable — every job has an id), `String title` (non-nullable, but only *assume* this if you trust the backend never omits it — otherwise `String?`), `double? salary` (nullable — the sample JSON literally has null), `List<String> tags` (non-nullable list, defaulting to `[]` if missing is safer than making the whole field nullable).

Chapter 6

Part 3 — Flutter Fundamentals: The Mental Model

This is the chapter that has no Spring Boot analogue, and it's the one most likely to make everything above it feel like arbitrary rules if you skip it. Read it slowly.

6.1 What Flutter actually is

Flutter is not a templating engine that fills in an HTML-like layout once. **Flutter re-runs a function that describes your entire UI, over and over, every time something might have changed, and calculates the minimum set of pixels that actually need to be redrawn.** That function is `build()`. Everything else in Flutter exists to control *when* `build()` runs and *what data it has available* when it does.

6.2 Widget, Widget Tree, Element Tree, Render Tree (beginner level)

- A **Widget** is not a UI element on screen. It is an immutable, lightweight **description** of what a piece of UI should look like right now — closer to a Java DTO than to an Android `View`. `Text('Hello')` is not “a hello text on screen”; it’s a small object holding the string `'Hello'` and some style info.
- The **Widget tree** is the nested structure of these description-objects — a **Scaffold** containing a **Column** containing a **Text** and a **Button**, exactly mirroring how you nest them in code.
- Every time `build()` runs, Flutter gets a **brand new widget tree** — entirely new objects. This is the part that surprises people coming from imperative UI frameworks: your old `Text('Hello')` object is thrown away and a new one is created, even if the text didn’t change.
- The **Element tree** is Flutter’s persistent bookkeeping layer *underneath* the widget tree. Elements are longer-lived than widgets — when a new widget tree comes in from a rebuild, Flutter compares it against the existing element tree (a process it calls “reconciliation,” conceptually similar to React’s virtual DOM diffing) and updates only what changed, reusing elements where the widget type and position match.
- The **Render tree** does the actual layout math (sizes, positions) and painting. You almost never touch this layer directly; it’s what turns “a Column with three children” into actual pixel coordinates.

You don’t need to manipulate elements or render objects directly for WorkBridge-level work — but knowing they exist explains *why* `const` widgets are a performance win (a `const` widget’s element can be reused without even comparing it) and why keys matter (covered below).

6.3 StatelessWidget vs StatefulWidget

```
class Greeting extends StatelessWidget {
  final String name;
  const Greeting({super.key, required this.name});

  @override
  Widget build(BuildContext context) {
    return Text('Hello, $name');
  }
}
```

A `StatelessWidget` has no memory of its own between rebuilds — every time it's rebuilt, it's rebuilt entirely from the data passed into its constructor by its parent. If `name` never changes, `Greeting` never needs to look different. This is the majority of `WorkBridge`'s small reusable pieces — `ErrorBanner`, `SuccessBanner`, `SectionHeader` in `widget/common_widget.dart` are all `StatelessWidgets`, because they have nothing to remember; they just render whatever they were told to render.

```
class Counter extends StatefulWidget {
  const Counter({super.key});
  @override
  State<Counter> createState() => _CounterState();
}

class _CounterState extends State<Counter> {
  int _count = 0; // THIS is the "state" - survives across rebuilds

  void _increment() {
    setState(() { _count++; }); // tells Flutter "rebuild me, something changed"
  }

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: _increment,
      child: Text('Count: $_count'),
    );
  }
}
```

A `StatefulWidget` is actually **two objects working together**: the `Counter` widget itself (immutable, thrown away and recreated on every rebuild, just like a `StatelessWidget`) and a separate `State<Counter>` object (`_CounterState`) that Flutter creates *once* and keeps alive across many rebuilds, specifically so it has somewhere to keep `_count` between them. This split is the single most important thing to understand about Flutter: **the widget is disposable, the State object is not.**

In `WorkBridge`, `_LoginScreenState` (from `login_screen.dart`) is a `State<LoginScreen>` holding `_formKey`, `_emailCtrl`, `_passwordCtrl`, `_loading`, `_errorMessage` — none of that data lives on `LoginScreen` itself, because `LoginScreen` gets thrown away and recreated on every rebuild, and if `_loading` lived there, it would reset to `false` every single rebuild, which is exactly wrong.

6.4 build() and BuildContext — what they actually mean

```
@override
Widget build(BuildContext context) { ... }
```

`build()` is not “run once when the screen appears.” It’s “run *every time* Flutter thinks this widget’s output might need to change” — after `setState()`, after a Riverpod-watched provider updates, after the parent widget rebuilds and passes new data down, sometimes even for reasons that have nothing to do with your own state (a parent’s `setState` can trigger a rebuild all the way down its subtree unless you optimize against it). Because of this, **`build()` must be cheap and side-effect-free** — no network calls, no heavy computation, nothing that shouldn’t happen dozens of times.

`BuildContext context` is a handle to *where this widget sits in the tree* at this moment. It’s how a widget finds things above it — the nearest `Theme`, the nearest `Navigator`, the nearest `MediaQuery` (screen size), or — critically for you — the nearest Riverpod `ProviderScope` when you call `ref.watch(...)` inside a `ConsumerWidget`. Java has no real equivalent; the closest analogy is a request-scoped context object that knows “which layer of the application am I currently inside,” except here it’s “which layer of the widget tree.”

6.5 setState() — what it actually triggers

```
setState() {
  _loading = true;
};
```

`setState()` does two things: (1) runs the callback you gave it, mutating your `State` object’s fields, and (2) tells Flutter “the output of this widget’s `build()` may now be different — schedule a rebuild.” **Mutating `_loading = true` outside of `setState()` would silently do nothing visible** — the field would change, but Flutter would have no idea it needs to redraw anything, because nothing told it to. This trips up almost everyone once.

In `WorkBridge`, notice `_login()`’s structure:

```
setState(() { _loading = true; _errorMessage = null; });
try {
  await ...;
  ...
} catch (e) {
  setState(() => _errorMessage = apiErrorMessage(e));
} finally {
  if (mounted) setState(() => _loading = false);
}
```

Three separate `setState` calls, each corresponding to a moment where the *visible* state of the screen genuinely changes: start loading, show an error, stop loading. This is the loading/error/success pattern you’ll see everywhere, and it’s built entirely out of `setState` here (screens that use Riverpod’s `AsyncValue` instead get this same pattern “for free” — see Part 12).

6.6 Widget Lifecycle

For a `StatefulWidget`, the important lifecycle methods, roughly in order:

```
createState()      - Flutter creates the State object, once
  ↓
initState()        - called once, right after the State object is inserted into the tree.
```

	Good place to kick off one-time setup (e.g. an initial data fetch)
↓	
build()	- called immediately after initState, and again after every setState() / dependency change, for the widget's whole life
↓ (repeats many times)	
dispose()	- called once, when the widget is permanently removed from the tree. MUST clean up anything you started in initState - controllers, subscriptions, timers

WorkBridge's dispose() discipline (from login_screen.dart):

```
@override
void dispose() {
  _emailCtrl.dispose();
  _passwordCtrl.dispose();
  super.dispose();
}
```

A `TextEditingController` holds native resources under the hood; forgetting to dispose it is a memory leak that accumulates every time the user navigates to and away from a screen. This is the Flutter equivalent of forgetting to close a `Connection` in JDBC — nothing crashes immediately, but it degrades the app over time. Notice `super.dispose()` is called *last* — the convention is “clean up your own stuff first, then let the framework finish.”

6.7 keys — a beginner-level answer to “why do widgets need keys?”

When Flutter reconciles a new widget tree against the old element tree, it matches widgets primarily by *type and position in their parent's children list*. This works fine until you reorder a list of widgets — without a `key`, Flutter might match “the third `ListTile`” to whatever *used to be* the third `ListTile`, even if the underlying data moved. A `Key` (usually `ValueKey(item.id)`) tells Flutter “match by this identity, not by position,” which matters most in `ListViews` where items can be added, removed, or reordered. You won't need this often at your current stage — WorkBridge's lists are mostly rebuilt wholesale from a fresh Riverpod fetch rather than reordered in place — but it's worth recognizing the symptom (list items visually “jumping” or showing stale content after a reorder) if you ever hit it.

6.8 Hot Reload vs Hot Restart

- **Hot reload** injects your changed code into the *already-running* app and re-runs `build()`, but keeps all existing `State` objects (and their field values) alive. Great for tweaking a widget's layout without losing your place in a form.
- **Hot restart** destroys the entire app state and starts over from `main()`. Necessary when you change something hot reload can't handle — `initState()` logic, global variables, enum definitions, or anything at the “top” of the app (like `main.dart` itself).

If you edit code and hot reload “does nothing” or behaves strangely, that's usually your signal to hot restart instead.

6.9 Exercises — Part 3

Level 1 — Recall 1. Why is a `Widget` described as a “lightweight, immutable description” rather than “a thing on screen”? 2. Why does `_CounterState` need to be a separate object from `Counter`? 3. What's the practical consequence of forgetting to call `.dispose()` on a `TextEditingController`?

Level 2 — Implementation 1. Write a `StatefulWidget` called `LikeButton` with a boolean `_liked` field, toggled by tapping an `IconButton`, using `setState`. 2. Add an `initState()` to a widget that prints `'screen opened'` to the console, and a matching `dispose()` that prints `'screen closed'`.

Level 3 — Problem Solving 1. You have a screen that fetches a user's notifications once when it first opens, then lets the user pull-to-refresh. Where does the *first* fetch belong — `build()` or `initState()`? Why would putting it in `build()` be a bug (think about how often `build()` runs)?

(Answers appear in “Exercise Answers: Part 3” at the end of Part 4.)

Chapter 7

Part 4 — Flutter UI and Layout

7.1 The constraint model, first — because it explains every “mistake” below

Flutter’s layout algorithm is a single rule, repeated recursively: **“a parent tells its child what constraints (min/max width, min/max height) it must fit inside; the child decides its own size within those constraints; the parent then positions the child.”** Constraints flow *down* the tree, sizes flow *up*. A widget can never just declare “I am 400px wide” in isolation — it can only pick a size within what its parent allowed.

This single rule explains almost every “why did I get an overflow error” moment:

- **Row/Column overflow (“A RenderFlex overflowed by N pixels”)**: a **Row** gives its children *unbounded* width to work with by default (each child can be as wide as it wants), but the **Row** itself is still constrained by its own parent. If the sum of the children’s natural widths exceeds what the **Row** was given, you get the classic yellow-and-black overflow stripes. Fix: wrap a child in **Expanded** or **Flexible** so it’s told to share the available space instead of taking its natural size, or wrap the whole **Row** in **SingleChildScrollView**.
- **“Vertical viewport was given unbounded height” inside a Column**: happens when you put an unconstrained widget like **ListView** directly inside a **Column** — a **Column** gives unbounded height to its children by default, and **ListView** wants to know a specific height to size itself against. Fix: wrap the **ListView** in **Expanded** (share remaining space) or **SizedBox(height: ...)** (fixed space).

Keep this rule in your head; it will save you more debugging time than memorizing any individual widget.

7.2 Widget Catalog

For each widget: what it’s for, the property that matters most, the mistake beginners make, and where (if anywhere) it appears in **WorkBridge**.

Widget	What it's for	Key property	Common mistake	WorkBridge example
MaterialApp	Root of the whole app — theme, routes, title	theme, onGenerateRoute	Forgetting it wraps everything — without it, most Material widgets (Scaffold , TextField) won't find the theming data they expect	main.dart 's MyApp.build()
Scaffold	Standard screen skeleton — app bar, body, FAB, drawer	body, appBar	Nesting a Scaffold inside another Scaffold unnecessarily	Every screen (login_screen.dart , etc.)
AppBar	Top bar with title/actions	title, actions	Forgetting it needs to be inside a Scaffold 's appBar: slot, not its body:	Used across list/detail screens
SafeArea	Pads content away from notches/status bars	child	Wrapping <i>everything</i> in SafeArea including things that should bleed to the edge (like a header banner)	—
Container	A box that can have color, padding, margin, decoration, size	decoration, padding	Using Container for simple padding when a plain Padding widget is cheaper/clearer	Login screen header block
SizeBox	An empty box of a fixed size, or a spacer	width/height	Using Container(height: 16) instead of the lighter SizeBox(height: 16) for pure spacing	Everywhere (SizeBox(height: 16))
Padding	Adds space around a single child	padding: EdgeInsets.all(24)	Forgetting EdgeInsets has .all , .symmetric , .only variants for asymmetric spacing	login_screen.dart form padding
Center	Centers its child within available space	child	Wrapping something in Center when the parent already centers it, doing nothing	Login screen's outer Center
Align	Like Center but with any alignment	alignment: Alignment.centerRight	—	"Forgot password?" link alignment

Widget	What it's for	Key property	Common mistake	WorkBridge example
Row	Lays children out horizontally	<code>mainAxisAlignment</code> , <code>children</code>	Overflow when children are wider than available space (see above)	—
Column	Lays children out vertically	<code>mainAxisSize</code> : <code>MainAxisSize.min</code>	Putting an unconstrained scrollable directly inside without Expanded	Login card's Column
Expanded	Forces a child to fill remaining Row/Column space	<code>flex</code>	Using it outside a Row/Column/Flex — only works as a direct child of one	—
Flexible	Like Expanded but child can be <i>smaller</i> than its share	<code>fit</code> : <code>FlexFit.loose</code>	Confusing it with Expanded — Expanded forces full fill, Flexible allows shrinking	—
Stack	Overlaps children on top of each other	<code>children</code> , positioned via Positioned	Forgetting Stack sizes itself to its <i>biggest</i> child by default, causing surprising overall sizes	Profile image + edit-icon overlay
Positioned	Places a child at an exact offset inside a Stack	<code>top</code> , <code>right</code> , etc.	Using it outside a Stack — compile error	Edit-icon badge on profile photo
Wrap	Like Row , but overflowing children wrap to a new line	<code>spacing</code> , <code>runSpacing</code>	Reaching for Row + manual overflow handling instead	Skill/tag chips
ListView	Scrollable, lazily-built list	<code>itemBuilder</code> (for <code>.builder</code>)	Using plain <code>ListView(children: [...])</code> for huge lists instead of <code>ListView.builder</code> , building every item up front	Job search results, gig search results
GridView	Scrollable grid	<code>gridDelegate</code>	Miscounting <code>crossAxisCount</code> for the target screen width	—
SingleChildScrollView	Makes a single, possibly-overflowing child scrollable	<code>child</code>	Wrapping a ListView in it (redundant — ListView is already scrollable, and nesting can break scroll physics)	Login form's outer scroll wrapper

Widget	What it's for	Key property	Common mistake	WorkBridge example
Card	A <code>Container</code> with elevation/rounded corners, Material-styled	<code>elevation</code> , <code>clipBehavior</code>	Forgetting <code>clipBehavior: Clip.antiAlias</code> when a child (like a colored header) needs to respect the rounded corners	Login card header
Text	Displays a string	<code>style</code>	Overriding style piecemeal instead of using <code>Theme.of(context).textTheme</code>	Everywhere
Icon	Displays a Material icon glyph	<code>icon</code> , <code>size</code> , <code>color</code>	—	Everywhere
Image	Displays an image (asset, file, or network — see Part 14)	<code>Image.network</code> , <code>Image.file</code> , <code>Image.asset</code>	Not handling the loading/error states of <code>Image.network</code>	Profile photos, company logos
CircleAvatar	Circular image/initial placeholder	<code>backgroundImage</code> , <code>child</code>	Forgetting a fallback <code>child</code> (initials) for when there's no image	Profile photo widgets
ElevatedButton	Primary filled button	<code>onPressed</code>	Passing <code>null</code> unintentionally, which <i>disables</i> the button (this is actually useful — see <code>LoadingButton</code> below)	Login/register buttons
TextButton	Low-emphasis text-only button	<code>onPressed</code>	—	“Forgot password?”, “Register” links
OutlinedButton	Medium-emphasis bordered button	<code>onPressed</code>	—	—
IconButton	A tappable icon	<code>onPressed</code> , <code>icon</code>	Forgetting <code>tooltip</code> for accessibility	Password visibility toggle
FloatingActionButton	Floating circular action button	<code>onPressed</code>	—	—
TextField	Raw text input, no <code>Form</code> integration	<code>controller</code> , <code>onChanged</code>	Using this instead of <code>TextFormField</code> when you actually need validation	—
TextFormField	Text input integrated with <code>Form</code> /validation	<code>validator</code> , <code>controller</code>	Forgetting to attach a <code>controller</code> and then not knowing how to read the value	Login/register forms

Widget	What it's for	Key property	Common mistake	WorkBridge example
Form	Groups <code>TextFormField</code> s for bulk validation	<code>key:</code> <code>GlobalKey<FormState>()</code>	Forgetting the <code>key</code> , which is how you trigger <code>.validate()</code> from outside	Login/register forms
DropDownButtonFormField	Dropdown integrated with Form	<code>items</code> , <code>onChanged</code>	Forgetting each <code>DropDownMenuItem</code> 's <code>value</code> must be unique and match the field's current value exactly (including type)	Role/employment-type selectors
Checkbox	Boolean toggle, square	<code>value</code> , <code>onChanged</code>	Forgetting <code>Checkbox</code> is “dumb” — it doesn't hold its own state, the parent must via <code>setState</code>	—
Radio	Single-select among options	<code>groupValue</code> , <code>onChanged</code>	Same statelessness trap as <code>Checkbox</code>	—
Switch	Boolean toggle, slider-style	<code>value</code> , <code>onChanged</code>	Same statelessness trap	—
Dialog/AlertDialog	Modal popup	<code>showDialog(context: ..., builder: ...)</code>	Using the <i>outer</i> <code>context</code> instead of the dialog's own inner <code>context</code> for <code>Navigator.pop()</code> inside the dialog	Confirmation dialogs
SnackBar	Transient bottom message	<code>ScaffoldMessenger.of(context).showSnackBar()</code>	Calling <code>context.showSnackBar()</code> or <code>Scaffold.of(context).showSnackBar()</code> when there's no <code>Scaffold</code> above in the same <code>context</code>	Confirmation dialogs
BottomSheet/showModalBottomSheet	Slide-up sheet	<code>builder</code>	Forgetting it needs its own <code>context</code> for <code>Navigator.pop()</code> , same trap as dialogs	—
Divider	Thin horizontal separator line	<code>height</code> , <code>thickness</code>	Confusing <code>height</code> (the total space reserved) with <code>thickness</code> (the visible line width)	<code>SectionHeader</code> widget

7.3 Building Screens: the Design → Code Process

The repeatable process, regardless of which screen you're building:


```
Design (what should it look like / do?)
↓
Identify the widgets that produce that look
↓
Sketch the widget tree (nesting) on paper first
↓
Build the static layout with placeholder/const data - no state yet
↓
Add State (StatefulWidget / ConsumerStatefulWidget) for anything that changes
↓
Wire up user interaction (onPressed, onChanged, form validation)
↓
Connect real data (Riverpod provider / repository call)
↓
Handle loading / error / success explicitly - never leave a screen with
    only the "happy path" wired up
```

The single biggest time-saver in this process: **build the static layout first, with fake data, before wiring any network calls.** It's much faster to fix a layout bug when you're not also debugging why an API call is failing, and vice versa. WorkBridge's screens follow this shape even if it's not visible in the final commit — the `Card` → `Column` → `Form` structure of `login_screen.dart` would have been built and visually verified *before* `_login()` was wired to `AuthController`.

7.4 Exercise Answers: Part 3

Level 1 1. Because the same `Widget` object describes UI that will be thrown away and rebuilt many times over the widget's "conceptual" lifetime on screen — it's a description, re-evaluated repeatedly, not a persistent screen object. 2. Because the widget itself is recreated on every rebuild, but something needs to persist data (like `_count`) *across* those rebuilds — that's the `State` object's entire job. 3. It leaks the native text-editing resources the controller holds — over many screen visits, this accumulates and can degrade app performance/memory over time.

Level 3 1. It belongs in `initState()`. `build()` can run dozens of times for reasons unrelated to "the screen just opened" — any `setState`, any parent rebuild, any dependency change. Fetching in `build()` would re-trigger the network call every single time, which is both wasteful and could cause visible flicker/race conditions as multiple overlapping requests resolve in unpredictable order.

Chapter 8

Part 5 — Building Screens: A Worked Example

Rather than repeat the process from Part 4 in the abstract, walk through building `LoginScreen` from nothing, in the order it was actually built.

Step 1 — static shell. Start with just `Scaffold(body: Center(child: Text('Login')))`. Confirm it runs.

Step 2 — the visual structure with fake content. Add the `Card` → `header Container` → `Column` nesting, with plain `TextFields` (not yet `TextFormField`) so you can see the layout without worrying about validation.

Step 3 — convert to a real Form. Swap `TextField` for `TextFormField`, wrap everything in `Form(key: _formKey, ...)`, add `validator` functions. At this point the screen *looks* finished but does nothing when submitted.

Step 4 — add local UI state. `_loading`, `_errorMessage`, `_showPassword` — become `StatefulWidget` fields, wired to `setState`.

Step 5 — connect real data. This is where `ConsumerStatefulWidget` (Riverpod’s version of `StatefulWidget` — see Part 12) and the actual `_login()` method wiring to `AuthController` happen.

Step 6 — handle every outcome, not just success. The `try/catch/finally` in `_login()`, the `ErrorBanner`, the `LoadingButton`’s disabled-while-loading state — this step is often skipped by beginners and it’s the difference between a demo and a shippable screen.

Other `WorkBridge` screens you can study with this same six-step lens: `register_screen.dart` (a longer form, same pattern), `job_details.dart` (a detail screen instead of a form — same “static shell → real data → loading/error/success” arc), `jobs_search_Screen.dart` (a list screen driven by a `FutureProvider`, see Part 12).

Chapter 9

Part 6 — Navigation

9.1 Navigator: push, pop, and friends

Flutter’s `Navigator` maintains a **stack** of routes (screens) — same mental model as a browser’s back/forward history, or an Android `Activity` back stack.

Method	What it does
<code>Navigator.push(context, route)</code>	Adds a new screen on top of the stack
<code>Navigator.pop(context)</code>	Removes the current screen, returning to the one below
<code>Navigator.pop(context, result)</code>	Same, but passes result back to whoever pushed this screen
<code>Navigator.pushReplacement(context, route)</code>	Replaces the <i>current</i> screen instead of stacking on top of it (used when the old screen shouldn’t be reachable via back)
<code>Navigator.pushNamedAndRemoveUntil(context, name, (route) => false)</code>	Pushes a screen and clears the <i>entire</i> stack beneath it — the <code>(route) => false</code> predicate means “remove everything”

9.2 Named Routes and WorkBridge’s `AppRouter`

Instead of constructing `MaterialPageRoute(builder: (_) => JobDetailsScreen(jobId: 15))` by hand at every call site, `WorkBridge` centralizes route names as string constants and a single dispatcher, `AppRouter.onGenerateRoute`:

```
class AppRouter {
  static const String jobDetails = '/job-details';
  ...
  static Route<dynamic> onGenerateRoute(RouteSettings settings) {
    switch (settings.name) {
      ...
      case jobDetails:
        final jobId = settings.arguments as int;
        return MaterialPageRoute(builder: (_) => JobDetailsScreen(jobId: jobId));
      ...
    }
  }
}
```

This is registered once in `main.dart` (`onGenerateRoute: AppRouter.onGenerateRoute`). Calling code anywhere in the app then just does:

```
Navigator.of(context).pushNamed(AppRouter.jobDetails, arguments: job.id);
```

Why centralize this at all? Two reasons that matter as your app grows: (1) route *names* become a single source of truth — rename `/job-details` once, and every call site that uses the constant `AppRouter.jobDetails` still compiles correctly, versus hunting down every hardcoded string; (2) arguments passing is centralized and cast in one place (`settings.arguments as int`), rather than every screen re-deriving how it was navigated to.

A worth-noting rough edge: `settings.arguments as int` is an unchecked cast — if some future call site accidentally passes a `String` instead of an `int`, this throws a runtime `TypeError` with a fairly generic message, not a compile-time error. This is the cost of `Navigator`’s untyped `arguments`: API; a routing package like `go_router` (which `WorkBridge` does *not* use — its own `AppRouter` is hand-rolled) offers typed route parameters that catch this class of mistake earlier. Worth knowing as a limitation, not a bug — for an app this size, the hand-rolled switch is a completely reasonable choice.

9.3 Returning Data From a Screen

```
// Screen A pushes screen B and waits for a result:
final result = await Navigator.of(context).push<bool>(
  MaterialPageRoute(builder: (_) => AddEditEducationScreen(educationId: null)),
);
if (result == true) {
  // refresh the list - something was added/edited
}

// Screen B, when done:
Navigator.of(context).pop(true);
```

This is `WorkBridge`’s actual convention (noted repeatedly across the CV sub-entity screens): an add/edit screen returns `true` via `Navigator.pop(true)` on success, and the list screen that pushed it uses that boolean as a signal to re-fetch. It’s a lightweight alternative to a shared “refresh” event bus — the list screen doesn’t need to know *what* changed, only *that* something did.

9.4 Authentication-Based and Role-Based Navigation

The pattern that actually decides “should this user see the login screen or the home screen” is **not** in `AppRouter` at all — it’s `AuthGate`, a `ConsumerWidget` sitting at the app’s root route:

```
class AuthGate extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final authState = ref.watch(authControllerProvider);
    return authState.when(
      loading: () => const Scaffold(body: Center(child: CircularProgressIndicator())),
      data: (user) => user != null ? const HomePage() : const LoginScreen(),
      error: (_, __) => const LoginScreen(),
    );
  }
}
```

Trace the flow: `AppRouter.root (/)` always resolves to `AuthGate`. `AuthGate` watches `authControllerProvider` — a Riverpod provider whose state is `AsyncValue<LoginResponse?>` (loading while it’s restoring a possibly-cached session, `data(user)` once resolved, `error` if something went wrong reading storage). Depending on that state, it shows a spinner, the home page, or the login screen — **without any `Navigator.push` at all.**

This is “declarative” auth-gating: rather than *imperatively* navigating to login when a token expires, the root of the app just re-renders based on current auth state, and every rebuild “re-decides” what should be showing.

Compare this to `login_screen.dart`’s success path, which *does* use `Navigator.pushNamedAndRemoveUntil('/', (route) => false)` — it doesn’t push straight to `HomePage`; it pushes back to root `/`, letting `AuthGate` re-evaluate and decide. This indirection matters: it means the login screen doesn’t need to know anything about roles or what the “home” screen even is — that decision is centralized in one place.

Role-based routing beyond authenticated-vs-not (e.g. showing a company dashboard vs a job-seeker dashboard) would extend this same pattern: `AuthGate` (or a widget just past it) reading `currentUserProvider`’s `.role` and branching to a different home widget per role, rather than scattering `if (role == UserRole.COMPANY)` checks across many screens.

Chapter 10

Part 7 — Forms and User Input

10.1 TextEditingController and FocusNode

A `TextEditingController` is the *bridge* between a `TextFormField`'s on-screen text and your Dart code — without one, you can still display an input, but you have no clean way to read its current value or set it programmatically.

```
final _emailCtrl = TextEditingController();
...
TextFormField(controller: _emailCtrl, ...)
...
final email = _emailCtrl.text.trim(); // read the current value at submit time
```

`FocusNode` tracks and controls which field currently has keyboard focus — useful for “move to the next field on submit” flows. WorkBridge’s forms mostly rely on `TextFormField`’s built-in `onFieldSubmitted` instead of manual `FocusNode` management (see `login_screen.dart`’s password field: `onFieldSubmitted: (_) => _login()`), which is a simpler, sufficient choice for forms this size.

10.2 Form, GlobalKey<FormState>, and Validation

```
final _formKey = GlobalKey<FormState>();
...
Form(
  key: _formKey,
  child: Column(children: [
    TextFormField(
      controller: _emailCtrl,
      validator: (v) {
        if (v == null || v.trim().isEmpty) return 'Enter a valid email.';
        if (!v.contains('@')) return 'Enter a valid email.';
        return null; // null = valid
      },
    ),
  ],
  ...
```

Each `TextFormField.validator` is called only when someone asks the `Form` to validate — `_formKey.currentState!.validate` as seen in `_login()`’s first line: `if (!_formKey.currentState!.validate()) return;`. Calling `.validate()` runs *every* field’s validator, shows their error messages inline for any that fail, and returns

`true` only if *all* of them passed. The `GlobalKey` is what lets code *outside* the `Form` widget (your `_login()` method) reach in and trigger this — it’s Flutter’s escape hatch for “control a descendant widget from an ancestor’s code,” used sparingly and mostly for exactly this case.

A validator returning `null` means “no error” — this is a common beginner trip-up, since `null` reads as “nothing happened” rather than “this passed,” but that’s exactly the contract `TextFormField` expects.

10.3 Disposing Controllers — non-negotiable

```
@override
void dispose() {
  _emailCtrl.dispose();
  _passwordCtrl.dispose();
  super.dispose();
}
```

Every controller you create in `initState()` (or as a field initializer) must be disposed in `dispose()`. This isn’t optional cleanup — skipping it is a real, accumulating resource leak, and Flutter’s own linter (`flutter_lints`, which WorkBridge has enabled in `pubspec.yaml`) will often flag missing disposal.

10.4 Password Fields, Dropdowns, Date Pickers, Image Selection

- **Password fields:** `obscureText: true` hides input; WorkBridge toggles this live with a `_showPassword` bool and an eye-icon `IconButton`, exactly as shown in `login_screen.dart`.
- **Dropdowns:** `DropdownButtonFormField<T>` integrates with `Form` the same way `TextFormField` does — its `value` must match one of its `items`’ values *by equality*, which is a common source of “dropdown shows nothing selected” bugs when comparing, say, an enum instance against a differently-constructed but logically-equal one.
- **Date pickers:** WorkBridge’s `personal_info_screen.dart` uses Flutter’s built-in `showDatePicker(context: ..., initialDate: ..., firstDate: ..., lastDate: ...)`, which returns a `Future<DateTime?>` — another real `await` in the wild, this time waiting on user interaction with a modal rather than a network call.
- **Image selection:** covered in full in Part 14, since it’s your named weak area — but the short version, straight from `personal_info_screen.dart`:

```
Future<void> _pickImage() async {
  final picked = await ImagePicker().pickImage(
    source: ImageSource.gallery, maxWidth: 1024, imageQuality: 85,
  );
  if (picked != null) setState(() => _pickedImage = File(picked.path));
}
```

10.5 A Complete Form Example: Registration

`register_screen.dart` extends everything above across more fields (email, password, confirm-password, full name, role selector), with the same shape: `GlobalKey<FormState>`, one `TextEditingController` per field, per-field validators, a loading flag, an error banner, and a submit handler that calls `.validate()` first before touching the network. If you can rebuild `LoginScreen` from memory, `RegisterScreen` is the same pattern with more fields — a good Level 2/3 exercise target (see Part 22).

Chapter 11

Part 8 — JSON and Models/DTOs

This is the part the instructions rightly flag as extremely important — get this wrong and every screen above it breaks in confusing, silent ways.

11.1 The Chain, In Both Directions

```
Backend JSON response
  ↓ jsonDecode()
Map<String, dynamic>    (Dart's untyped, generic view of the JSON)
  ↓ YourDTO.fromJson(map)
Typed Dart DTO object  (compiler now enforces field types)
  ↓
UI reads typed fields - dto.title, dto.salary, etc.
```

and in reverse, for anything you send:

```
Typed Dart DTO / request object
  ↓ dto.toJson()
Map<String, dynamic>
  ↓ jsonEncode() (Dio does this for you automatically for `data:`)
JSON text
  ↓
HTTP request body
```

Dio does the `jsonDecode/jsonEncode` steps for you — when you call `_dio.post(url, data: dto.toJson())`, Dio serializes the map to a JSON string and sets the `Content-Type: application/json` header automatically; when a response comes back, `response.data` is *already* a `Map<String, dynamic>` (or `List<dynamic>`), not a raw string. This is why `WorkBridge`'s `fromJson` factories always take a `Map<String, dynamic>` directly — the decoding already happened before your code sees it.

11.2 `fromJson()` in Detail — Using `LoginResponse` as the Case Study

```
factory LoginResponse.fromJson(Map<String, dynamic> json) {
  return LoginResponse(
    token: json['token'] as String?,
    tokenType: json['tokenType'] as String? ?? 'Bearer',
```



```

    userId: (json['userId'] as num?)?.toInt(),
    email: json['email'] as String?,
    role: UserRole.fromJson(json['role'] as String?),
    profileId: (json['profileId'] as num?)?.toInt(),
    displayName: json['displayName'] as String?,
    image: json['image'] as String?,
  );
}

```

`json['token']` reads the value at key `'token'` from the map — but its *declared* type at that point is `dynamic` (Dart’s “I have no idea, figure it out yourself” type), because `Map<String, dynamic>` says nothing about what’s actually inside. `as String?` is a **cast**: “trust me, this is a `String?`” — if the backend actually sent a number under that key, this cast throws a `TypeError` at runtime, with no compile-time warning at all. This is the single biggest practical risk of hand-written JSON parsing (versus Java’s Jackson, which fails at a similarly “runtime” point but with better error messages, or a compile-time-checked DTO mapper): **a mismatch between your Dart field’s assumed type and what the backend actually sends is invisible until that exact code path runs.**

`UserRole.fromJson(json['role'] as String?)` shows nested parsing — a field that’s itself another type with its own `fromJson`, rather than a primitive.

11.3 toJson() in Detail

```

Map<String, dynamic> toJson() {
  return {
    if (token != null) 'token': token,
    'tokenType': tokenType,
    if (userId != null) 'userId': userId,
    ...
  };
}

```

The `if (token != null) 'token': token` syntax is that same collection-if from Part 1, applied to a map literal — it means “only include this key at all if the value isn’t null,” rather than including it as `'token': null`. This matters because many REST APIs (including yours) distinguish between “field omitted” and “field explicitly set to null” for PATCH-style partial updates — omitting a key says “don’t touch this field,” while sending `null` can mean “clear this field.” WorkBridge’s `toJson()` methods lean toward omission, which is the safer default for most request DTOs.

11.4 Nullable Fields — Why APIs Force `String?`, `int?`, `DateTime?`

Three real reasons a field ends up nullable in a Dart DTO, all present somewhere in WorkBridge:

1. **The backend field is genuinely optional** — e.g. `salary` on a job posting that hasn’t set one.
2. **You can’t fully trust the contract** — even a required backend field *could* theoretically arrive as `null` due to a bug, a schema migration, or an old cached response; making the Dart field nullable is defensive.
3. **The value doesn’t exist yet in this DTO’s lifecycle** — a request DTO you’re building up field-by-field in a multi-step form before submission (see `job_search_request.dart`’s `JobSearchRequestDTO`, which has many optional filter fields, all nullable, since a search with no filters set is valid).

11.5 Nested JSON and Lists

```
factory JobResponseDTO.fromJson(Map<String, dynamic> json) {
  return JobResponseDTO(
    id: json['id'] as int,
    title: json['title'] as String,
    company: json['company'] != null
      ? CompanyResponseDTO.fromJson(json['company'] as Map<String, dynamic>)
      : null,
    ...
  );
}
```

A nested object becomes a nested `fromJson` call, passed the inner map. A JSON array becomes a `List`, parsed with `.map()`:

```
Future<List<JobResponseDTO>> searchJobs(JobSearchRequestDTO request) async {
  final response = await _dio.post(ApiConstants.searchJobs, data: request.toJson());
  return (response.data as List)
    .map((json) => JobResponseDTO.fromJson(json))
    .toList();
}
```

`response.data as List` casts the raw response to a Dart `List<dynamic>` — each element is still an untyped `Map<String, dynamic>` at that point. `.map((json) => JobResponseDTO.fromJson(json))` runs `fromJson` once per element, producing an `Iterable<JobResponseDTO>`; `.toList()` materializes it into an actual `List<JobResponseDTO>`. This exact three-line shape — cast to `List`, `.map(fromJson)`, `.toList()` — appears throughout WorkBridge’s repositories (`job_repository.dart`’s `searchJobs`, `getApplicationsByUserProfileId`, and equivalents in `gig`, `wallet`, `chat`) and is worth memorizing as *the* pattern for “parse a JSON array response.”

Chapter 12

Part 9 — REST API Fundamentals (From the Client Side)

You already understand REST from writing `@RestController`s. What's new is *consuming* one — deciding, for each piece of data you need to send, whether it's a path parameter, a query parameter, a request body field, or a header.

Mechanism	Looks like	When WorkBridge uses it
Path parameter	<code>/jobs/15</code>	Identifying <i>one specific resource</i> — <code>ApiConstants.getJobUrl(id)</code> => <code>'jobs/\$id'</code>
Query parameter	<code>/jobs/search?keyword=java&page=0</code>	Optional filters/pagination on a <i>collection</i> endpoint — though notice WorkBridge's job/gig search actually sends filters as a <i>JSON body</i> via POST (see below), not query params, which is a deliberate choice for complex multi-field filters
JSON request body	<code>{"keyword": "java", "page": 0}</code>	Anything structured/multi-field being <i>created or searched</i> — login credentials, registration data, search filters
Header	<code>Authorization: Bearer <JWT></code>	Cross-cutting metadata that applies to the <i>whole request</i> , not specific to the resource — auth tokens, content type

Why does WorkBridge's job search use a POST with a JSON body instead of GET with query params, even though it's conceptually a “read” operation? Look at `JobSearchRequestDTO` — it likely has many optional fields (keyword, location, job type, salary range, employment type, pagination). Encoding a dozen optional filters as query-string parameters gets unwieldy and is easy to get wrong (URL encoding of special characters, arrays-in-query-strings ambiguity); a JSON body sidesteps all of that at the cost of technically bending REST convention (POST is “supposed to” create something). This is a common, pragmatic real-world tradeoff — plenty of production search APIs do exactly this, sometimes calling the endpoint `/jobs/search` specifically to signal “this reads, despite the verb.”

12.1 Status Codes You’ll Actually Handle

Your `apiErrorMessage()` function (in `utils/api_client.dart`) already encodes the ones that matter:

```
switch (status) {  
  case 401: return 'Your session has expired. Please log in again.';  
  case 403: return 'Your account is not verified or has been disabled.';  
  case 404: return 'Not found.';  
  case null: return 'Could not reach the server. Check your connection / API URL.';  
  default: return 'Something went wrong. Please try again.';  
}
```

Notice `case null` — that’s not “the server returned a null status code,” it’s “there was no response at all” (`error.response` itself is `null`), which happens for network failures, DNS errors, timeouts, or a completely unreachable server — distinct from the server actively responding with an error status. This distinction (no response vs. an error response) is one you’ll want to preserve in any error-handling code you write yourself.

Chapter 13

Part 10 — Dio

13.1 Why Dio, when http is also a dependency

`pubspec.yaml` lists both `http` and `dio`. `http` is Dart's minimal, low-level HTTP client — you'd write your own JSON encoding, header management, and interceptor-equivalent logic by hand. `dio` builds much more on top: interceptors (critical for JWT — see Part 11), automatic JSON encode/decode, typed exceptions (`DioException`) instead of generic ones, configurable timeouts, and first-class multipart support (critical for Part 14). WorkBridge's actual network calls all go through `dio`; `http` being present as a dependency without being the primary client is common in real projects (sometimes left over from an earlier prototype, sometimes used for one specific edge case elsewhere) — worth noting, not a red flag.

13.2 Setting Up a Dio Instance: `BaseOptions`

```
dio = Dio(  
  BaseOptions(  
    baseUrl: ApiConstants.baseUrl,  
    connectTimeout: const Duration(seconds: 2000),  
    receiveTimeout: const Duration(seconds: 2000),  
  ),  
);
```

`baseUrl` means every call site can use a relative path (`'jobs/search'`) instead of the full URL — this is *why* `ApiConstants` stores paths like `static const String login = 'auth/login'`; without a scheme or host. `connectTimeout/receiveTimeout` set how long Dio waits before giving up. (Worth flagging honestly: 2000 seconds — over 33 minutes — is an unusually generous timeout, almost certainly a placeholder left over from development/debugging rather than a production-tuned value; a real-world default is closer to 10–30 seconds, with per-request overrides for genuinely slow operations like large file uploads.)

13.3 The Five Methods

```
await dio.get('/jobs/15');  
await dio.post('/jobs', data: request.toJson());  
await dio.put('/jobs/15', data: request.toJson());  
await dio.patch('/jobs/15/status', data: {'active': false});  
await dio.delete('/jobs/15');
```

Same semantics as the Spring Boot `@GetMapping/@PostMapping/etc.` you already write on the server: GET

reads without side effects, POST creates (or, as above, executes a read-with-body search), PUT replaces a resource wholesale, PATCH updates part of one, DELETE removes it. WorkBridge's `job_repository.dart` uses `patch` for `withdrawApplication` — a targeted state change (withdrawing, not replacing the whole application), which is exactly what PATCH is for.

13.4 queryParameters, data, headers

```
final response = await dio.get(
  '/jobs/search',
  queryParameters: {'keyword': 'Java', 'page': 0},
);
```

Dio appends these as `?keyword=Java&page=0` and handles URL-encoding for you.

```
final response = await dio.post('/jobs', data: request.toJson());
```

`data`: becomes the JSON request body; Dio serializes the `Map<String, dynamic>` automatically.

```
await dio.get('/private', options: Options(headers: {'X-Custom': 'value'}));
```

Per-request headers via `Options`. WorkBridge doesn't need this much, because the JWT header — the one header that matters on almost every call — is attached globally by the interceptor (next section), not per-call.

13.5 Response and DioException

Every successful call returns a `Response` object: `response.data` (the parsed body), `response.statusCode`, `response.headers`. Every failed call *throws* a `DioException` (not returns an error object — this is a real throw, caught with `try/catch` or on `DioException` catch (`e`)), carrying `e.response` (may be `null` if there was no response at all — see Part 9), `e.type` (a `DioExceptionType` enum distinguishing timeouts, cancellations, bad responses, etc.), and `e.message`.

13.6 Interceptors — WorkBridge's ApiClient, Explained Line by Line

```
dio.interceptors.add(
  InterceptorsWrapper(
    onRequest: (options, handler) async {
      final token = await _storageService.getToken();
      if (token != null && token.isNotEmpty) {
        options.headers['Authorization'] = 'Bearer $token';
      }
      handler.next(options);
    },
    onError: (error, handler) async {
      if (error.response?.statusCode == 401) {
        await _handleUnauthorized();
      }
      handler.next(error);
    },
  ),
);
```

An **interceptor** is a function that Dio runs on *every* request/response/error passing through this Dio instance — it's the same architectural idea as a Spring **HandlerInterceptor** or a servlet **Filter**, just on the client instead of the server.

- **onRequest** runs *before* every outgoing request leaves the app. This one reads the stored JWT (**await** — reading from secure storage is itself **async**) and, if present, sets the **Authorization** header. **This is the entire reason no individual repository method in WorkBridge ever manually attaches a token** — **AuthRepository**, **JobRepository**, **CvRepository**, all of them just call **_dio.get(...)/_dio.post(...)** and trust that if a token exists, it's already on the request by the time it's sent.
- **handler.next(options)** is mandatory — it tells Dio “I’m done modifying this request, send it.” Forgetting to call it hangs the request forever, since Dio is waiting for this interceptor to explicitly hand control back.
- **onError** runs when a request comes back as a failure. Checking specifically for 401 (Unauthorized — the JWT is missing, invalid, or expired) triggers **_handleUnauthorized()**, which clears the stored session and force-navigates to the login screen via **NavigationService** — explored fully as the centerpiece of Part 11.
- **handler.next(error)** re-propagates the error onward so the original caller's **try/catch** still sees it — the interceptor's job here is a *side effect* (logging out), not swallowing the error.

13.7 Multipart Requests — Preview

Briefly, because Part 14 covers this in full: Dio represents a **multipart/form-data** body as a **FormData** object, with **MultipartFile.fromFile(path)** / **MultipartFile.fromBytes(bytes)** for the file parts and **MultipartFile.fromString(json)** for JSON metadata sent alongside a file in the same request — exactly what **WorkBridge**'s **saveUserProfile()** does when uploading a profile with an optional photo in one call.

Chapter 14

Part 11 — JWT Authentication: The Complete Flow

You wrote the server side of this — the `AuthController`, the JWT filter, the `@PreAuthorize` checks you audited across 41 controllers. This chapter is the *other half*: what the Flutter client does with the token your backend issues.

14.1 The Full Chain

```
Flutter login form (LoginScreen)
  ↓ user taps "Sign In"
LoginRequest(email, password)
  ↓ dto.toJson()
Dio POST auth/login
  ↓
Spring Boot: authenticates, issues JWT
  ↓
JSON response: { token, tokenType, userId, email, role, ... }
  ↓ LoginResponse.fromJson(response.data)
AuthRepository.login() also does: await _storageService.saveSession(loginRes)
  ↓
StorageService writes token → flutter_secure_storage (ENCRYPTED, unlike shared_preferences)
  ↓
AuthController's state becomes AsyncValue.data(loginRes)
  ↓
Every widget watching authControllerProvider rebuilds - AuthGate now shows HomePage
  ↓ ... later, any authenticated request ...
ApiClient's onRequest interceptor reads the stored token
  ↓
Authorization: Bearer <JWT> header attached automatically
  ↓
Spring Boot's JWT filter validates it, authorizes the request
  ↓
Response returns normally
```


14.2 Why flutter_secure_storage and Not shared_preferences

`shared_preferences` stores data in plain text (in Android’s `SharedPreferences` XML, or equivalent per-platform storage) — readable by anyone with access to the device’s filesystem on a rooted/jailbroken device, or via backup extraction on some configurations. `flutter_secure_storage` uses the platform’s actual secure storage (Android Keystore-backed encryption, iOS Keychain) — the same category of protection a password manager uses. A JWT is effectively a bearer credential — anyone holding it can act as the user until it expires — so it belongs in the more protected store. WorkBridge is deliberate about this split: `StorageService` uses `FlutterSecureStorage` exclusively for the token and cached user object, while `shared_preferences` remains a dependency available for non-sensitive local data (though it’s worth noting it’s not clear it’s actually used anywhere yet in the current codebase — a dependency present but not yet exercised is common and not itself a problem).

14.3 Storing and Restoring the Session

```
Future<void> saveSession(LoginResponse data) async {  
  await _storage.write(key: StorageKeys.token, value: data.token);  
  await _storage.write(key: StorageKeys.user, value: jsonEncode(data.toJson()));  
}
```

Two things get stored: the raw token string, and the *entire user object* re-serialized to JSON text (secure storage is key-value, string-to-string — there’s no native “store an object” concept, so `jsonEncode/jsonDecode` bridge that gap the same way it does for HTTP bodies).

```
Future<void> _restoreSession() async {  
  final storage = _ref.read(storageServiceProvider);  
  final user = await storage.getUser();  
  state = AsyncValue.data(user);  
}
```

This runs once, from `AuthController`’s constructor, every time the app cold-starts. It’s what lets a user close the app and reopen it later *still logged in* — the token was never re-requested from the server, it was simply read back out of secure storage. This is also exactly why `AuthGate` needs its loading state (`const AsyncValue.loading()` is `AuthController`’s starting state) — reading from secure storage is itself async, so there’s a brief real window where the app genuinely doesn’t yet know if the user is logged in.

14.4 Attaching the Token — the Interceptor, Again

Already covered mechanically in Part 10; the JWT-specific point worth restating here: **not one single feature module (job, gig, wallet, cvinformations, chat) contains any code that reads the token or sets an Authorization header.** That responsibility lives in exactly one place — `ApiClient`’s constructor. This is the payoff of centralizing it: if your backend ever changes the header format (say, dropping the Bearer prefix, or renaming the header), there’s exactly one line to change, not dozens scattered across every repository.

14.5 Logout

```
Future<void> logout() async {  
  await _ref.read(authRepositoryProvider).logout();  
  state = const AsyncValue.data(null);  
}
```

`AuthRepository.logout()` is just `_storageService.clearSession()` — deletes both stored keys. Setting

`state = const AsyncValue.data(null)` is what actually drives the UI: `AuthGate.build()` re-evaluates, sees `data(null)`, and shows `LoginScreen` instead of `HomePage`. Notice **there is no `Navigator.push` anywhere in logout** — same declarative pattern as login, just in reverse: change the state, let `AuthGate` react.

14.6 Expired JWT Handling — the `_handleUnauthorized()` Flow

```
onError: (error, handler) async {
  if (error.response?.statusCode == 401) {
    await _handleUnauthorized();
  }
  handler.next(error);
},

Future<void> _handleUnauthorized() async {
  if (_loggingOut) return;
  _loggingOut = true;
  try {
    await _storageService.clearSession();
    final navigator = NavigationService.navigatorKey.currentState;
    if (navigator != null) {
      navigator.pushNamedAndRemoveUntil(AppRouter.login, (route) => false);
    }
  } finally {
    _loggingOut = false;
  }
}
```

This is the piece that fires when the *server* rejects the token — expired, revoked, or otherwise invalid — on any request, from any screen, at any time. A few details worth understanding, because this is genuinely more advanced than most beginner tutorials cover:

- **`_loggingOut` is a guard flag.** If a screen fires several requests in quick succession right as the token expires (a common real scenario — a list screen might fetch data + counts + images near-simultaneously), *multiple* 401s could arrive close together, each independently trying to trigger a logout-and-redirect. Without this flag, you’d get redundant, possibly UI-thrashing redirect attempts. The flag ensures only the *first* 401 actually acts; the rest see `_loggingOut == true` and return immediately.
- **`NavigationService.navigatorKey` exists** because the interceptor’s `onError` runs deep inside Dio, with no `BuildContext` available at all — interceptors aren’t widgets, they have no natural access to the widget tree. `NavigationService` sidesteps this by keeping a global `GlobalKey<NavigatorState>` that’s attached to the app’s root `MaterialApp` (`navigatorKey: NavigationService.navigatorKey` in `main.dart`) precisely so that non-widget code — an interceptor — can still trigger navigation. This is a deliberate, common pattern for exactly this situation, not a workaround to be embarrassed about.
- **`pushNamedAndRemoveUntil(AppRouter.login, (route) => false)`** clears the *entire* back stack, not just pushes login on top — you don’t want the user hitting “back” from the login screen and landing on a now-unauthorized screen that will just immediately 401 again.

14.7 Exercises — Part 11

Level 1 — Recall 1. Why does the token get stored in `flutter_secure_storage` rather than `shared_preferences`? 2. What problem does the `_loggingOut` flag specifically prevent?

Level 2 — Implementation 1. Add a `Future<bool> isValidSession()` method to `StorageService` that returns `true` only if a token exists *and* is non-empty (reuse the existing `getToken()`).

Level 3 — Problem Solving 1. Suppose your backend adds token *refresh* — a short-lived access token plus a longer-lived refresh token, and an endpoint to exchange the refresh token for a new access token before it expires. Sketch (in prose, no code needed) where in **ApiClient** this refresh logic would need to live, and why it would need to happen inside the interceptor rather than in each repository method.

(Answers: see “Exercise Answers: Parts 11–13” after Part 13.)

Chapter 15

Part 12 — Riverpod: Taught Properly, Not “It Just Manages State”

This is the part of Flutter that has no clean Java analogue, so it deserves the slowest treatment in this book. Don’t skip the “why” section for the “how” — the how will feel arbitrary without it.

15.1 The Problem `setState` Stops Solving

`setState()` (Part 3) works fine when the data that changed belongs to *exactly one* widget’s private state. It falls apart the moment two things are true at once, both of which are true throughout `WorkBridge`:

1. **Multiple, unrelated widgets need the same piece of data.** `currentUserProvider`’s value (who’s logged in) is needed by `AuthGate`, by the home page’s app bar, by screens that need `userId` to fetch a profile, and more. With plain `setState`, you’d have to manually pass this data down through every constructor between wherever it lives and wherever it’s needed — “prop drilling” — and manually notify every listener when it changes.
2. **The data outlives any single screen.** The logged-in user’s identity needs to survive navigating between a dozen different screens. A `State` object tied to one `StatefulWidget` dies the moment that widget leaves the tree; it can’t hold app-wide data.

Riverpod solves both: it lets you declare a piece of state **once, at the top level, independent of any specific widget**, and any widget anywhere in the tree can *watch* it — get the current value, and automatically rebuild when it changes — without any constructor plumbing.

15.2 The Core Idea: a Provider is a Recipe, Not a Value

```
final apiClientProvider = Provider<ApiClient>((ref) {  
  return ApiClient(ref.watch(storageServiceProvider));  
});
```

`apiClientProvider` is not an `ApiClient`. It’s a **description of how to create one**, plus (implicitly) a place Riverpod will cache the result once it’s created. The first time anything asks for it, Riverpod runs the callback, builds the `ApiClient`, and remembers it — every subsequent ask gets the *same cached instance*, not a new one. This is dependency injection: exactly what Spring’s `ApplicationContext` does with a

@Bean method, except Riverpod resolves the dependency graph the first time it's *used* rather than eagerly at startup.

`ref.watch(storageServiceProvider)` inside another provider's definition is how providers depend on each other — `apiClientProvider` needs a `StorageService`, so it *watches* `storageServiceProvider` to get one, forming a chain. WorkBridge's `auth/providers.dart` shows the whole small dependency graph explicitly:

```
final secureStorageProvider = Provider<FlutterSecureStorage>((ref) => const FlutterSecureStorage());
final storageServiceProvider = Provider<StorageService>((ref) => StorageService(ref.watch(secureStorageProvider)));
final apiClientProvider = Provider<ApiClient>((ref) => ApiClient(ref.watch(storageServiceProvider)));
```

Three links: `FlutterSecureStorage` → `StorageService` → `ApiClient`. Change how `StorageService` is constructed, and every provider depending on it (directly or transitively) automatically gets the updated version — you never manually thread it through constructors app-wide.

15.3 ProviderScope and Where the Graph Actually Lives

```
void main() {
  runApp(const ProviderScope(child: MyApp()));
}
```

`ProviderScope` is the container that actually holds every provider's cached value — it's the Riverpod equivalent of Spring's `ApplicationContext` instance. Wrapping the whole app in exactly one `ProviderScope` (as WorkBridge does) means there's one shared graph for the entire app's lifetime. Without a `ProviderScope` above it anywhere in the tree, `ref.watch(...)` has nothing to ask and will throw.

15.4 The Provider Types WorkBridge Actually Uses

Type	Holds	WorkBridge example	When to reach for it
<code>Provider<T></code>	A value/object that doesn't change after creation (a singleton/dependency)	<code>apiClientProvider</code> , <code>authRepositoryProvider</code> , <code>jobRepositoryProvider</code>	Dependency injection — repositories, services, clients
<code>StateProvider<T></code>	A simple mutable value, replaced wholesale via <code>.state = newValue</code>	<code>jobSearchFilterProvider</code> (holds the current search filter object)	A small piece of UI-controlled state with no async loading involved
<code>FutureProvider<T></code>	The result of one async operation, auto-wrapped as <code>AsyncValue<T></code>	<code>jobSearchResultsProvider</code> , <code>categoriesProvider</code> , <code>myUserProfileProvider</code>	“Fetch this once (or once per dependency change) and give me loading/data/error automatically”
<code>StateNotifierProvider<Notifier, State></code>	<code>Notifier</code> (implements <code>StateNotifier</code>) with real <i>methods</i> that update its own state over time	<code>authControllerProvider</code> (backed by <code>AuthController</code> extends <code>StateNotifier<AsyncValue<LoginResponse>></code>)	State that needs <i>behavior</i> attached to it, not just a value — login/logout, not just “login response”

WorkBridge does **not** currently use `AsyncNotifier/Notifier` (Riverpod's newer replacement for `StateNotifier`) or `StreamProvider` anywhere in the codebase — `StateNotifier` (via the `flutter_riverpod/legacy.dart` import you'll notice at the top of `auth_provider.dart` and `job_provider.dart`) is the pattern actually in use. Worth knowing explicitly: this means WorkBridge is written against an older, “legacy” corner

of Riverpod 3.x’s API (the newer `Notifier/AsyncNotifier` classes are now the officially recommended replacement) — not wrong, and it works identically for your purposes, but if you look at newer Riverpod tutorials online and see `Notifier` instead of `StateNotifier`, that’s why they look different from your code.

15.5 `AsyncValue<T>` — Loading/Success/Error, Unified

Every `FutureProvider` and every `StateNotifierProvider<_, AsyncValue<T>>` exposes its state as an `AsyncValue<T>` — a single type representing exactly three possibilities: still loading, resolved with data, or failed with an error. You almost never manually check “am I loading” as a separate boolean; you pattern-match on the whole thing at once:

```
resultsAsync.when(
  loading: () => const Center(child: CircularProgressIndicator()),
  error: (e, _) => Center(child: Text('Failed to load jobs: $e')),
  data: (jobs) => ListView.builder(itemCount: jobs.length, ...),
)
```

This is `jobs_search_Screen.dart`, close to verbatim. `.when()` forces you to handle *all three* cases — the compiler won’t let you provide only `data:` and forget `loading:/error:`, which is precisely the discipline the instructions call out: “handle loading, error, and success explicitly, never leave a screen with only the happy path wired up.” `AsyncValue` makes skipping that a compile error instead of a runtime surprise.

Notice the retry button in that same screen:

```
OutlinedButton(
  onPressed: () => ref.invalidate(jobSearchResultsProvider),
  child: const Text('Retry'),
)
```

`ref.invalidate(...)` throws away a provider’s cached value and forces it to rebuild from scratch the next time it’s watched — here, re-running the search. This is the cleanest way to implement “try again” for a `FutureProvider`: you don’t need a separate refresh method, invalidation *is* the refresh mechanism.

15.6 `ConsumerWidget`, `ConsumerStatefulWidget`

You cannot call `ref.watch(...)` from a plain `StatelessWidget/StatefulWidget` — there’s no `ref` available. Riverpod provides parallel widget base classes that hand you one:

```
class AuthGate extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) { // note the extra `ref` param
    final authState = ref.watch(authControllerProvider);
    ...
  }
}
```

`ConsumerWidget` is the `StatelessWidget` equivalent (no local mutable state of its own, but *can* watch providers). `ConsumerStatefulWidget/ConsumerState<T>` is the `StatefulWidget` equivalent — used when a screen needs *both* its own local `setState`-managed fields (like `LoginScreen`’s `_loading`, `_emailCtrl`) *and* access to providers. Look again at `_LoginScreenState`:

```
class LoginScreen extends ConsumerStatefulWidget { ... }
class _LoginScreenState extends ConsumerState<LoginScreen> {
  ...
  await ref.read(authControllerProvider.notifier).login(...);
}
```

Here `ref` is available as a member of `ConsumerState`, not a `build()` parameter — same capability, different access point depending on which base class you’re extending.

15.7 `ref.watch` vs `ref.read` vs `ref.listen` — Why You Can’t Use `ref.watch` Everywhere

This is the single most common real Riverpod mistake, so it’s worth being precise:

- **`ref.watch(provider)`** — “give me the current value, **and rebuild this widget whenever it changes.**” Use this inside `build()`, for anything the UI needs to *display*.
- **`ref.read(provider)`** — “give me the current value **right now, once, and don’t subscribe to future changes.**” Use this inside callbacks (`onPressed`, `_login()`) where you’re *triggering an action*, not rendering — subscribing to changes there would be meaningless (the callback already finished running by the time the value could change) and, worse, calling `ref.watch` outside `build()` is simply not what it’s for and can cause subtle bugs or errors depending on context.
- **`ref.listen(provider, (previous, next) { ... })`** — “run this side-effect callback whenever the value changes, but *don’t* rebuild this widget because of it.” Used for things like showing a `SnackBar` in reaction to a state change, where you want the *reaction* but not a rebuild triggered by watching.

Concretely, in `_login()`: `ref.read(authControllerProvider.notifier).login(...)` — this is inside a button-press callback, calling a *method* on the controller (`.notifier` gets you the `StateNotifier` instance itself, as opposed to its current state value), so `read` is correct; there’s nothing to “watch” here, you’re issuing a command. Meanwhile `AuthGate.build()` uses `ref.watch(authControllerProvider)` — reading the *state*, and needing to rebuild whenever it changes (loading → data). Same provider, two different access patterns, chosen for two different purposes.

15.8 Every Real Provider in WorkBridge, Explained

Provider	State it owns	Who reads it	Who modifies it	Why it exists
<code>secureStorageProvider</code>	FlutterSecureStorage instance	<code>storageServiceProvider</code>	Nobody (never changes after creation)	Single shared secure-storage handle
<code>storageServiceProvider</code>	<code>StorageService</code> instance	<code>apiClientProvider</code> , <code>authRepositoryProvider</code> , <code>AuthController</code>	Nobody	Central session read/write API
<code>apiClientProvider</code>	A configured Dio-wrapping <code>ApiClient</code> (with interceptors attached)	Every repository provider	Nobody	One Dio instance, one interceptor setup, shared everywhere
<code>authRepositoryProvider</code>	<code>AuthRepository</code>	<code>AuthController</code>	Nobody	Auth-specific network calls
<code>authControllerProvider</code>	<code>AsyncValue<LoginResponse></code> — who’s logged in, or not	<code>AuthGate</code> , <code>LoginScreen</code> , anything needing the current user	<code>AuthController.login()</code> , <code>.logout()</code> , <code>._restoreSession()</code>	The single source of truth for “is anyone logged in, and as whom”
<code>currentUserProvider</code>	<code>LoginResponse?</code> (derived — just unwraps <code>authControllerProvider.value</code>)	Screens needing <code>userId/role</code> without caring about its loading state	Nobody directly — it follows <code>authControllerProvider</code>	Convenience: skip re-writing <code>when(...)</code> everywhere just to get the user

Provider	State it owns	Who reads it	Who modifies it	Why it exists
<code>jobSearchFilterProvider</code>	The current <code>JobSearchRequestDTO</code> filter	The search screen's filter UI, <code>jobSearchResultsProvider</code>	The search/filter UI, via <code>provider.state = newFilter</code>	Lets filter UI and results be decoupled — the results provider reacts automatically
<code>jobSearchResultsProvider</code>	<code>AsyncValue<List<JobSearchResult>></code>	The search screen's <code>.when()</code>	Nobody directly — re-runs automatically whenever <code>jobSearchFilterProvider</code> changes (because it <code>ref.watches</code> it)	Search-as-you-filter without manual “refetch” plumbing
<code>myUserProfileProvider</code> / <code>myCompanyProfileProvider</code>	The signed-in user's own profile, <code>null</code> if not yet created	Profile screens, <code>ProfileGate</code>	Nobody directly — re-fetches when <code>currentUserProvider</code> changes	Distinguishes “no profile yet” (backend 404, treated as <code>null</code>) from a genuine failure (rethrown, becomes a real error)

That last row is worth a closer look — it's a genuinely nice piece of error-handling design:

```
final myUserProfileProvider = FutureProvider<UserProfileResponseDTO?>((ref) async {
  final userId = ref.watch(currentUserProvider)?.userId;
  if (userId == null) return null;
  try {
    return await ref.watch(cvRepositoryProvider).getUserProfileByUserId(userId);
  } on DioException catch (e) {
    if (e.response?.statusCode == 404) return null;
    rethrow;
  }
});
```

A 404 here doesn't mean “something went wrong” — it means “this user hasn't filled out a profile yet,” a legitimate, expected state that the UI should treat as “show a create-profile prompt,” not “show an error banner.” Catching `DioException` specifically, checking the status code, and only *rethrowing* for anything else, is how this provider distinguishes an expected empty state from a genuine failure — both of which would otherwise collapse into the same generic `AsyncValue.error` if left unhandled.

15.9 Exercises — Part 12

Level 1 — Recall 1. Why can't `setState` alone hold app-wide data like “who's logged in”? 2. What's the practical difference between `ref.watch` and `ref.read`? 3. What does `.when()` force you to provide, and why is that a good constraint?

Level 2 — Implementation 1. Write a `StateProvider<bool>` called `showOnlyActiveJobsProvider`, defaulting to `true`. 2. Write a `FutureProvider<int>` called `jobCountProvider` that depends on `jobSearchResultsProvider` and returns its length (hint: `ref.watch` the results provider, then use `.value?.length ?? 0` or handle it via `.when`).

Level 3 — Problem Solving 1. You're adding a “saved jobs” (bookmark) feature. Sketch which provider

type each of these needs, and why: (a) the list of the user’s saved job IDs, fetched from the backend once and cached; (b) toggling a single job’s saved/unsaved status, which needs a method, not just a value; (c) whether a *specific* job (by id) is currently saved, computed from (a) for use in a list item’s icon.

(Answers appear in “Exercise Answers: Parts 11–13” after Part 13.)

Chapter 16

Part 13 — Repository Pattern and Application Architecture

16.1 The Layers, and Why Each One Exists

```
Screen (Widget)
  ↓ reads via ref.watch / calls via ref.read(...).notifier
Controller / Provider (StateNotifier, or a plain FutureProvider)
  ↓ calls repository methods
Repository
  ↓ calls _dio.get/post/...
ApiClient (Dio + interceptors)
  ↓ HTTP
Spring Boot API
```

- **Why not call Dio directly from a widget?** Because the widget would then own three unrelated responsibilities at once: describing UI, parsing JSON, and knowing your API's URL scheme. Change your backend's response shape, and you'd be hunting through UI code to fix parsing logic. This is the exact same reasoning you already apply on the backend when you keep `@RestControllers` thin and push logic into `@Service` classes.
- **Why do repositories exist, specifically, as a layer separate from the provider/controller?** A repository's *only* job is “translate between Dio's HTTP world and typed Dart DTOs” — `JobRepository.searchJobs()` takes a typed request, returns a typed response, and knows nothing about Riverpod, loading states, or UI at all. This makes repositories trivially testable in isolation (Part 17) and reusable — nothing stops two different providers from calling the same repository method for different purposes.
- **Why do DTOs exist** rather than passing raw `Map<String, dynamic>` around? Type safety, auto-complete, and — critically — a single place (`fromJson`) where a backend contract mismatch surfaces, instead of scattered `map['someKey']` lookups throughout the UI that each silently return `null` if the key is misspelled.
- **Why do providers “provide” dependencies at all**, instead of repositories just constructing their own `ApiClient`? So that the *entire dependency chain* is swappable and centrally managed — in a test, you can override `apiClientProvider` with a fake and every repository built on top of it transparently uses the fake, with zero changes to repository code itself (Part 17 shows this concretely).

16.2 Tracing Three Real WorkBridge Features End to End

16.2.1 Feature 1 — Logging In

```
LoginScreen (_login())
  ↓ ref.read(authControllerProvider.notifier).login(email, password)
AuthController.login() - sets state = loading, calls repo, sets state = data/error
  ↓ _ref.read(authRepositoryProvider).login(LoginRequest(...))
AuthRepository.login()
  ↓ _dio.post(ApiConstants.login, data: dto.toJson())
  ↓ LoginResponse.fromJson(res.data)
  ↓ await _storageService.saveSession(loginRes) ← side effect: persist token
ApiClient.dio (interceptor attaches nothing here - no token exists yet pre-login)
  ↓ HTTP POST
Spring Boot AuthController → issues JWT
```

16.2.2 Feature 2 — Searching Jobs

```
JobsSearchScreen (filter UI changes)
  ↓ ref.read(jobSearchFilterProvider.notifier).state = newFilter
jobSearchFilterProvider (StateProvider - just holds the new filter object)
  ↓ (jobSearchResultsProvider is WATCHING this provider)
jobSearchResultsProvider (FutureProvider) automatically re-runs
  ↓ ref.watch(jobRepositoryProvider).searchJobs(filter)
JobRepository.searchJobs()
  ↓ _dio.post(ApiConstants.searchJobs, data: request.toJson())
  ↓ (response.data as List).map(JobResponseDTO.fromJson).toList()
ApiClient.dio (interceptor attaches the JWT - this endpoint likely requires auth)
  ↓ HTTP POST
JobsSearchScreen's resultsAsync.when(...) rebuilds with the new list
```

Notice there is no explicit “refetch” call anywhere in this trace — changing the filter is enough. This is the reactive-graph payoff of Riverpod: `jobSearchResultsProvider`’s dependency on `jobSearchFilterProvider` (via `ref.watch` *inside the provider definition itself*, not just inside a widget) means the provider graph does the “when should I refetch” reasoning for you.

16.2.3 Feature 3 — Uploading a Profile Photo

```
PersonalInfoScreen (_pickImage())
  ↓ ImagePicker().pickImage(...) → picked file path → setState(_pickedImage = File(path))
  ↓ ... user taps Save ...
  ↓ (calls a CvRepository method directly via ref.read, or via a provider - see Part 19)
CvRepository.saveUserProfile(request, imageFile)
  ↓ builds FormData: one part = JSON string (request.toJsonString()), one part = the image file
  ↓ _dio.post(ApiConstants.userProfiles, data: formData)
ApiClient.dio (interceptor attaches JWT; Dio sets Content-Type: multipart/form-data automatically
  because the body is a FormData object, not a plain map)
  ↓ HTTP POST, multipart body
Spring Boot controller reads two parts: the JSON metadata + a MultipartFile
```

This third trace is deliberately the shortest here because Part 14 walks through every line of it.

16.3 Exercise Answers: Parts 11–13

Part 11, Level 1 1. `flutter_secure_storage` encrypts its contents using the platform’s secure storage (Android Keystore / iOS Keychain); `shared_preferences` stores plain text, which is a meaningfully weaker guarantee for something as sensitive as a bearer token. 2. It prevents multiple near-simultaneous 401 responses (from several in-flight requests) from each independently triggering a redundant logout-and-redirect sequence.

Part 11, Level 3 The refresh logic would live inside the `onError` interceptor callback (alongside, or instead of, the current “if 401, log out” branch): on a 401, instead of immediately clearing the session, attempt one call to a refresh endpoint using the stored refresh token; if that succeeds, retry the original failed request with the new access token via `handler.resolve(await dio.fetch(retriedOptions))`; if the refresh itself fails, *then* fall through to the existing clear-session-and-redirect behavior. It has to live in the interceptor (not in each repository method) for the same reason JWT attachment does — it needs to apply uniformly to every request without every repository knowing or caring that refresh logic exists.

Part 12, Level 1 1. `setState` only affects the one `State` object it’s called on, and that object dies with its widget — it has no mechanism for a *different, unrelated* widget elsewhere in the tree to be notified or to read the value at all. 2. `ref.watch` subscribes to future changes and causes a rebuild when the value changes (used in `build()`); `ref.read` reads the current value once, without subscribing (used in callbacks/one-off actions). 3. It forces you to handle `loading`, `data`, and `error` — you cannot compile code that only handles the happy path, which prevents the common bug of a screen that silently does nothing (or crashes) while a request is in flight or has failed.

Part 12, Level 2 — sample solutions:

```
final showOnlyActiveJobsProvider = StateProvider<bool>((ref) => true);

final jobCountProvider = FutureProvider<int>((ref) async {
  final jobs = await ref.watch(jobSearchResultsProvider.future);
  return jobs.length;
});
```

(`.future` on a `FutureProvider` gives you an awaitable `Future<T>` version of it — handy when you want to depend on the *resolved* value from inside another async provider, rather than pattern-matching an `AsyncValue`.)

Part 12, Level 3 (a) `FutureProvider<List<int>>` (or `Set<int>`) — a fetch-once, cacheable async list, same shape as `myUserProfileProvider`. (b) A `StateNotifierProvider` (or, in modern Riverpod, an `AsyncNotifierProvider`) — because toggling needs a *method* (`toggleSaved(jobId)`) with real logic (call the backend, then update local state optimistically or refetch), not just a value replacement. (c) A plain `Provider` (or just a `.family`-parameterized derived provider) that reads (a)’s current data and checks `.contains(jobId)` — it doesn’t need its own network call or its own state, it’s purely computed from (a).

Chapter 17

Part 14 — File and Image Handling (Your Named Weak Area — Slow, Thorough Treatment)

17.1 Selecting an Image: `image_picker`

```
Future<void> _pickImage() async {
  final picked = await ImagePicker().pickImage(
    source: ImageSource.gallery,
    maxWidth: 1024,
    imageQuality: 85,
  );
  if (picked != null) setState(() => _pickedImage = File(picked.path));
}
```

`ImagePicker().pickImage()` opens the OS's native photo picker (or camera, with `source: ImageSource.camera`) and returns a `Future<XFile?>` — `null` if the user cancelled, an `XFile` otherwise. `XFile` is `image_picker`'s cross-platform wrapper around “a file that was picked” — on mobile it has a real filesystem `path`; the reason it's `XFile` and not a plain `dart:io File` is that on **Flutter Web there is no filesystem at all**, so a bare `File` can't represent a picked image there. `maxWidth/imageQuality` ask the OS to downscale/compress *before* handing the file back, which matters for upload size and speed. `File(picked.path)` converts the cross-platform `XFile` into a concrete `dart:io File` — this line would actually throw or misbehave on web, which is exactly why this pattern only appears in `personal_info_screen.dart` (targeting mobile use directly) rather than in code paths that need to run on web too.

17.2 Selecting a File: `file_picker`

```
final pickedFile = await FilePicker.pickFile(
  type: FileType.custom,
  allowedExtensions: ['.pdf', '.docx'],
);
```

This is your actual resume-upload flow (`resume_file_screen.dart`). `type: FileType.custom` plus `allowedExtensions` restricts the OS file browser to only showing `.pdf/.docx` files. The returned object (here wrapped by a small `FilePicker.pickFile` helper — in raw `file_picker` this is

`FilePicker.platform.pickFiles(...)`, returning a `FilePickerResult` with a `.files` list) exposes both a `path` (mobile/desktop) *and* a way to read raw `bytes` — and this dual availability is the crux of the next section.

17.3 Why File Alone Isn’t Enough: the Web/Mobile Split

```
if (kIsWeb) {
  final bytes = await pickedFile.readAsBytes();
  uploaded = await repo.uploadResume(profileId, null, bytes: bytes, fileName: pickedFile.name);
} else {
  if (pickedFile.path != null) {
    uploaded = await repo.uploadResume(profileId, File(pickedFile.path!));
  } else {
    // Fallback to bytes if path is null (e.g. cloud file on mobile)
    final bytes = await pickedFile.readAsBytes();
    uploaded = await repo.uploadResume(profileId, null, bytes: bytes, fileName: pickedFile.name);
  }
}
```

This is genuinely one of the trickier, more advanced pieces of code in your whole project, and it’s worth unpacking fully because the reasoning generalizes to any file-upload feature you build later:

- **On Flutter Web**, there is no OS filesystem the Dart code can address — a picked file exists only as **in-memory bytes**, handed to your code by the browser. `pickedFile.path` is `null` on web *by design*, not by bug. So the web branch unconditionally reads bytes.
- **On mobile**, a picked file *usually* has a real `path` you can hand straight to `MultipartFile.fromFile(path)`. But the inner `else` comment — *// Fallback to bytes if path is null (e.g. cloud file on mobile)* — flags a real edge case: some mobile file sources (a file picked from Google Drive/iCloud rather than local storage) may hand back content without a resolvable local path, so this code defensively falls back to reading bytes there too.
- The repository method (`uploadResume`) accepts *either* a `File?` *or* `bytes: Uint8List? + fileName: String?`, precisely so a single repository method can serve both code paths:

```
Future<ResumeFileResponseDTO> uploadResume(int userProfileId, File? file, {Uint8List? bytes, String? fileName}) {
  final formData = FormData.fromMap({
    if (file != null)
      'cv': await MultipartFile.fromFile(file.path, filename: file.path.split(Platform.pathSeparator).last),
    else if (bytes != null)
      'cv': MultipartFile.fromBytes(bytes, filename: fileName ?? 'resume.pdf'),
  });
  ...
}
```

`MultipartFile.fromFile(path)` streams a file straight from disk. `MultipartFile.fromBytes(bytes)` builds the same multipart part from an in-memory byte array instead — same wire format, different source. This is the general-purpose lesson: **any code you write that “gets a file from the user” and might run on web needs a bytes-based fallback path**, because `dart:io.File` simply does not exist as a usable filesystem handle on web.

17.4 Uploading a File — the Full Chain

```
Flutter: File or bytes picked
↓
MultipartFile.fromFile(path) or MultipartFile.fromBytes(bytes)
```

```

↓
FormData.fromMap({ 'cv': multipartFilePart, ['userprofile': jsonMetadataPart] })
↓
Dio sends this as an HTTP request with Content-Type: multipart/form-data,
each part separated by a boundary string Dio generates automatically
↓
Spring Boot receives it - the 'cv' part maps to a controller parameter
typed org.springframework.web.multipart.MultipartFile,
exactly the class name you already know from the backend

```

A **multipart/form-data** body is fundamentally different from a plain JSON body: instead of one blob of JSON text, it's a sequence of named “parts,” each with its own headers, some containing text/JSON and some containing raw binary file data, all packed together with boundary markers. **You can't simply send a File as JSON** because JSON is a text format with no native way to represent arbitrary binary bytes — you'd have to Base64-encode the file into a string first, which inflates the payload by roughly a third and is exactly the inefficiency multipart encoding avoids. This is why `saveUserProfile()` (Part 10's preview) sends the JSON metadata as *one multipart part* (`MultipartFile.fromString(request.toJsonString(), contentType: DioMediaType('application', 'json'))`) alongside the image as a second part, rather than as a query parameter or a separate request — a single multipart request can carry both structured data and a file together.

17.5 Displaying Images: the Five Cases

Source	Widget	Example
Bundled with the app	<code>Image.asset('assets/icon/workbridge.png')</code>	The app logo, which icon source (declared in <code>pubspec.yaml</code> 's <code>assets:</code> list)
A file just picked, not yet uploaded	<code>Image.file(_pickedImage!)</code> (or <code>FileImage</code> inside a <code>CircleAvatar.backgroundImage</code>)	Preview of a profile photo before saving
Already on the server, public	<code>Image.network(url)</code> (or <code>NetworkImage</code> inside <code>CircleAvatar</code>)	Existing profile photo, company logo
Already on the server, requires auth	Image.network does not attach headers by default — this needs special handling (below)	—
Cached network image	Not currently a dependency — <code>WorkBridge</code> doesn't use <code>cached_network_image</code>	—

17.6 URL Construction for Files

```

static String get fileUrl => 'http://$host:8090/api/files/';
static String get userProfileImageUrl => '${fileUrl}userprofiles/';

```

The pattern is always **base URL + resource-type segment + filename**, matching `ApiConstants`'s structure exactly: `baseUrl (.../api/)` is for JSON endpoints; `fileUrl (.../api/files/)` is a *separate* base specifically for raw file serving, and each file category gets its own sub-path (`userprofiles/`, `resumes/`, `portfolios/`, `trainings/`, `messages/`, `gigdeliveries/`, `gigs/`). Displaying an existing profile image is then just:

```
Image.network('${ApiConstants.userProfileImageUrl}${user.image}')
```

17.7 Authenticated Network Images — A Real Gap Worth Knowing About

`Image.network(url)` performs a plain GET request with no custom headers — it cannot attach your JWT the way Dio’s interceptor does, because it doesn’t go through Dio at all; it uses Flutter’s own lower-level networking. In `personal_info_screen.dart` this is visibly worked around by using `NetworkImage(_existingImageUrl!)` directly for an *existing* photo — which only works because WorkBridge’s file-serving endpoints are apparently public/unauthenticated reads (reasonable for profile photos, which typically aren’t sensitive). **If you ever need to display a genuinely access-controlled image** (a private document, a paid-content thumbnail), `Image.network` alone won’t work — the real solution is either (a) fetch the bytes yourself via your authenticated Dio instance (`_dio.get(url, options: Options(responseType: ResponseType.bytes))`) and feed them into `Image.memory(bytes)`, or (b) have the backend issue short-lived signed URLs that don’t require the `Authorization` header at all. This is a genuinely intermediate-to-advanced Flutter topic, worth knowing exists rather than needing to have memorized yet.

17.8 Downloading and Opening Files

```
Future<Uint8List> getResumePdf(int userProfileId) async {
  final response = await _dio.get<List<int>>(
    ApiConstants.resumePdf(userProfileId),
    options: Options(responseType: ResponseType.bytes),
  );
  return Uint8List.fromList(response.data ?? []);
}
```

`responseType: ResponseType.bytes` tells Dio “don’t try to parse this as JSON, give me raw bytes” — necessary for binary responses like a generated PDF. This *does* go through your authenticated Dio instance, so the JWT is attached — contrast this with the `Image.network/launchUrl` approaches below, which bypass Dio entirely and therefore only work for unauthenticated file URLs.

```
Future<void> _viewUploadedResume() async {
  final fileName = _uploadedResume?.fileName;
  final url = Uri.parse('${ApiConstants.resumeFileUrl}${fileName}');
  final opened = await launchUrl(url, mode: LaunchMode.externalApplication);
  if (!opened && mounted) {
    ScaffoldMessenger.of(context).showSnackBar(const SnackBar(content: Text('Could not open resume.')))
  }
}
```

`url_launcher`’s `launchUrl` hands the URL off to the *operating system* — on mobile this typically opens the file in an external viewer app or the browser, rather than rendering it inside your Flutter app. `LaunchMode.externalApplication` explicitly requests this (as opposed to trying to open it in an in-app webview). `launchUrl` returns a `bool` — `true` if the OS found something able to handle the URL — and this code checks it, showing a `SnackBar` rather than silently doing nothing if opening fails. This is the “opening/viewing” answer for WorkBridge specifically: it delegates PDF/document *viewing* to the OS/browser rather than embedding an in-app PDF viewer widget, which is a simpler, lower-maintenance choice at this project’s stage.

17.9 File Paths, Bytes, MIME Types, Extensions, Content-Type — Quick Reference

- **File path:** a filesystem address (`/data/user/0/.../cache/image123.jpg`), meaningful only on platforms with a real filesystem — mobile/desktop, not web.
- **Bytes:** the raw binary content itself, platform-independent, always available regardless of whether a path exists.
- **MIME type / Content-Type:** a string like `image/jpeg` or `application/pdf` telling the receiver how to interpret the bytes — `MultipartFile.fromFile/.fromBytes` can infer this from the extension or accept an explicit override (as `saveUserProfile()` does for its JSON part: `contentType: DioMediaType('application', 'json')`), because that part *isn't* the file itself).
- **File extension:** the human-facing hint (`.pdf`, `.jpg`) — `file_picker`'s `allowedExtensions` filters by this, but the actual MIME type sent over the wire is a separate, explicit concept the multipart part carries.

17.10 Exercises — Part 14

Level 1 — Recall 1. Why does `pickedFile.path` come back null on Flutter Web? 2. Why can't you just send a picked File as a field inside a plain JSON body?

Level 2 — Implementation 1. Write a function `Future<void> pickAndPreviewPortfolioImage()` that uses `ImagePicker`, stores the result in a `File?` field via `setState`, following the exact pattern in `personal_info_screen.dart`.

Level 3 — Problem Solving 1. You're adding a “company logo” upload, and it needs to work on both mobile and Flutter Web, matching the resume-upload dual-path pattern. Sketch (in prose) the branching logic you'd need in the picking code, and what the repository method's signature should look like to support both paths in one method.

(Answers appear in “Exercise Answers: Parts 14–17” after Part 17.)

Chapter 18

Part 15 — Web vs Android vs iOS: Why “localhost” Lies to You

18.1 ApiConstants.host, Explained

```
static String get host {  
  if (kIsWeb) return 'localhost';  
  switch (defaultTargetPlatform) {  
    case TargetPlatform.android: return '10.0.2.2';  
    case TargetPlatform.iOS: return 'localhost';  
    default: return 'localhost';  
  }  
}  
  
static String get baseUrl => 'http://$host:8090/api/';
```

Here’s the problem this solves. You’re running your Spring Boot backend on your development machine, listening on `localhost:8090`. Depending on *where* your Flutter app is actually executing, “localhost” refers to a completely different machine from your perspective:

- **Flutter Web:** the app runs *inside your desktop browser*, on the same machine as your backend. `localhost` genuinely means “this computer” — correct as-is.
- **iOS Simulator:** the simulator shares the host Mac’s network stack closely enough that `localhost` also correctly resolves to your development machine.
- **Android Emulator:** this is the one that surprises everyone the first time. The emulator runs as its own virtual machine with its **own** network namespace — inside the emulator, `localhost` means “the emulator itself,” not your host computer. Google’s Android emulator specifically maps the special IP `10.0.2.2` to “the host machine’s `localhost`” — it’s not your backend’s real IP, it’s a fixed alias the emulator provides for exactly this situation.
- **A real physical device** (not covered by this switch, worth knowing as a gap): neither `localhost` nor `10.0.2.2` would work — a real phone on your Wi-Fi network needs your development machine’s actual LAN IP address (e.g. `192.168.1.42`), because the phone is a genuinely separate device on the network, not a VM sharing the host’s loopback interface. If you ever test on a physical device, `ApiConstants.host` as currently written would need a third branch (or an environment-variable/build-flavor override) for this case.

This is the single most common “why can’t my app reach my backend” confusion for anyone starting Flutter development against a local server, and `WorkBridge’s ApiConstants.host` getter exists entirely to paper over it with one function instead of forcing every developer to remember and manually swap URLs.

18.2 CORS on Flutter Web

CORS (Cross-Origin Resource Sharing) is a **browser-only** security mechanism — it has no bearing on the Android/iOS builds at all, because native apps aren't subject to the same-origin policy a browser enforces. On Flutter Web, though, your app runs *as* a web page, served from one origin (say `http://localhost:PORT` where your Flutter dev server serves it), making requests to a *different* origin (`http://localhost:8090`, your Spring Boot server) — different port counts as a different origin. Unless your Spring Boot backend explicitly sends **Access-Control-Allow-Origin** headers permitting this, the browser will block the response from ever reaching your Dart code, even though the request technically succeeded on the wire — you'd see a **DioException** with no real response, and the actual CORS rejection reason visible only in the browser's own developer console, not in anything Dio surfaces directly.

18.3 File Access and Download Differences

Covered concretely in Part 14, but the general principle: **web has no filesystem your Dart code can address directly** — `dart:io.File` doesn't function there at all, which is why file-picking code needs the `kIsWeb` branch to fall back to in-memory bytes, and why a “download” on web typically means triggering the browser's own download mechanism rather than writing to a path your app controls.

Chapter 19

Part 16 — Error Handling and Debugging

19.1 A Systematic Process (use this instead of guessing)

```
Something's wrong
↓
Read the ENTIRE error message, not just the first line
↓
Find the exact file and line number the error points to
↓
Understand what type/value was EXPECTED at that point
↓
Add a print() (or breakpoint) right before that line to inspect the ACTUAL value
↓
If it's a network call: check the request in Dio's logs / a tool like Proxyman,
    then check whether the backend even received it, then check the backend's own logs
↓
Fix the ROOT cause - not just the symptom
    (e.g. don't just wrap a crashing line in try/catch and swallow it -
     understand WHY the value was unexpected in the first place)
```

19.2 Categories of Errors You'll Actually Hit

- **Compile errors:** the tooling (flutter analyze, your IDE) catches these before you even run the app — a missing `required` argument, a type mismatch. These are the *cheapest* bugs to fix because Dart's static typing already caught them for you; take advantage of this by not ignoring red squiggles.
- **Widget/layout errors** (the yellow-and-black overflow stripes, "RenderFlex overflowed," "Vertical viewport was given unbounded height"): almost always resolved by re-reading the constraint model from Part 4 — ask "what widget is giving this child unbounded space it can't handle?"
- **Null errors** (Null check operator used on a null value — from a stray `!`, or `NoSuchMethodError: The method 'x' was called on null`): these mean a value you assumed was non-null (usually via `!`) was actually `null` at runtime. The fix is almost never "just add another `!`" — it's tracing *why* the value was null when you expected it not to be, which is very often a JSON parsing mismatch (Part 8) or a race condition (an `awaited` value used before it resolved).
- **DioException/API errors:** check `e.response?.statusCode` first — `null` means no response reached you at all (network/CORS/wrong host — see Part 15); a real status code means the server responded

but rejected the request, and `e.response?.data` usually contains the backend’s actual error message, which is your fastest path to the real cause.

- **JSON parsing / type errors** (`type 'Null'` is not a subtype of `type 'String'`, `type 'int'` is not a subtype of `type 'String'`): almost always a mismatch between what a `fromJson` assumed (`json['x'] as String`) and what the backend actually sent for that key (missing, or a different JSON type like a number). Print `response.data` raw, before parsing, to see exactly what arrived.
- **Provider/state errors** (“No `ProviderScope` found,” “Bad state: tried to use a provider outside of a widget,” `ref.watch` called somewhere it shouldn’t be): almost always either (a) missing the `ProviderScope` wrapper at the app root, or (b) calling `ref.watch` inside a callback/`initState`-style location instead of `build()` (use `ref.read` there instead — Part 12).

19.3 Why `ListView` Sometimes Causes Layout Errors, `NoSuchMethodError`, and Provider Rebuilds — Answered Together

These three questions from the FAQ (Part 21) are really one underlying idea each — repeated briefly here because they’re common enough to deserve a place in the debugging workflow directly: `ListView` errors trace back to the constraint model (Part 4); `NoSuchMethodError` on JSON almost always traces back to `fromJson` mismatches (Part 8); a provider “rebuilds too often” almost always traces back to watching a provider that changes more often than the widget actually needs to react to (e.g. watching the whole `AsyncValue` when you only need `.value`, or watching a provider inside a widget that’s larger than it needs to be, so unrelated parts of the UI re-render along with the part that actually depends on the changed data).

Chapter 20

Part 17 — Testing (Fundamentals, Kept Practical)

20.1 The Three Levels

- **Unit tests:** test a single function/class in isolation — e.g. does `LoginResponse.fromJson()` correctly parse a known JSON map? Does `UserRole.fromJson('admin')` correctly return `UserRole.ADMIN` regardless of case?
- **Widget tests:** test that a widget renders/behaves correctly in a simulated environment, without a real device — e.g. does `LoginScreen` show a validation error when submitted empty?
- **Integration tests:** test a real flow end-to-end, typically against a real (or realistically faked) backend, on a real or simulated device — e.g. does the full login → home page → logout flow work?

20.2 Mocking

A **mock** is a fake stand-in for a real dependency, letting you test one layer without needing the layer below it to actually work (or exist, or be reachable over a network during a test run). This is where Riverpod's dependency-injection design pays off directly:

```
test('AuthRepository.login parses a successful response', () async {  
  // A fake ApiClient whose Dio instance is configured with a mock adapter  
  // that returns a canned response instead of making a real HTTP call  
  final mockDio = Dio()..httpClientAdapter = _FakeAdapter(  
    responseJson: {'token': 'abc', 'userId': 1, 'email': 'a@b.com', 'role': 'USER'},  
  );  
  final fakeApiClient = ApiClient(fakeStorageService)..dio = mockDio;  
  
  final repo = AuthRepository(fakeApiClient, fakeStorageService);  
  final result = await repo.login(LoginRequest(email: 'a@b.com', password: 'x'));  
  
  expect(result.token, 'abc');  
  expect(result.role, UserRole.USER);  
});
```

This is roughly the same idea as mocking a `RestTemplate/WebClient` in a Spring `@Test` — you're not testing that Dio can make HTTP calls (that's the library's job to have already tested), you're testing that *your* parsing and repository logic behaves correctly given a known response.

20.3 Testing a Provider

Riverpod supports **overriding** any provider for the duration of a test, using a fresh `ProviderContainer` instead of the real app's `ProviderScope`:

```
test('jobSearchResultsProvider returns jobs from the repository', () async {
  final container = ProviderContainer(
    overrides: [
      jobRepositoryProvider.overrideWithValue(_FakeJobRepository()),
    ],
  );
  final jobs = await container.read(jobSearchResultsProvider.future);
  expect(jobs, isEmpty);
});
```

`overrideWithValue` swaps out the real `JobRepository` for a fake one *only inside this test's `ProviderContainer`* — everything downstream (`jobSearchResultsProvider`, which depends on `jobRepositoryProvider`) automatically uses the fake without any change to its own code. This is precisely the payoff described in Part 13 for having a separate DI layer at all: swappability without touching business logic.

Keep this practical: for a project at WorkBridge's current stage, a handful of unit tests around your trickiest `fromJson/toJson` pairs and your `ApiClient`'s error-message logic will catch more real regressions per hour invested than a large widget-testing suite would — start there if you start anywhere.

20.4 Exercise Answers: Parts 14–17

Part 14, Level 1 1. Flutter Web has no filesystem the browser exposes to your Dart code — a picked file only ever exists as in-memory bytes handed over by the browser, so there is no local path for it to have. 2. JSON is a text format with no native binary representation — sending file bytes would require Base64-encoding them into a string first, which both bloats the payload (~33% larger) and is exactly the inefficiency multipart encoding exists to avoid; multipart lets binary bytes travel as their own raw part instead.

Part 14, Level 3 Picking logic: check `kIsWeb` first — on web, always read bytes via `readAsBytes()` and pass `bytes:/fileName;`; off web, prefer `pickedFile.path` when non-null (`MultipartFile.fromFile`), falling back to bytes if `path` is unexpectedly null (cloud-sourced file). Repository signature: `Future<CompanyProfileResponseDT0>` `uploadLogo(int companyId, File? file, {Uint8List? bytes, String? fileName})` — mirroring `uploadResume`'s exact shape, building a `FormData` part conditionally from whichever of `file/bytes` was provided.

Chapter 21

Part 18 — Build a Mini WorkBridge From Scratch: “TaskBridge”

Do not open your real WorkBridge project for this part. The goal is to prove you can produce this shape of code without looking at an existing implementation — that’s the actual skill this whole book is building toward. Set up a fresh Flutter project (`flutter create taskbridge`), add `dio`, `flutter_riverpod`, `flutter_secure_storage`, `image_picker`, and build a tiny app against *any* simple REST backend that supports login + a list + a detail + one file upload (your own WorkBridge Spring Boot backend works fine, pointed at a couple of its simpler endpoints, or a placeholder backend of your choosing).

For every feature below: read the concept, **attempt it yourself before reading the solution**, then compare.

21.1 1–2. Login + JWT Authentication

Concept: a Form collects credentials, a repository posts them and receives a token, the token is stored securely, and a global auth-state provider reflects the result.

Your turn: build `LoginRequest`, `LoginResponse` (with `fromJson/toJson`), a `TaskBridgeApiClient` with an `onRequest` interceptor attaching a stored token, an `AuthRepository.login()`, and an `AuthController` extends `StateNotifier<AsyncValue<LoginResponse?>>`.

Solution sketch:

```
class AuthController extends StateNotifier<AsyncValue<LoginResponse?>> {
  AuthController(this._ref) : super(const AsyncValue.loading()) { _restore(); }
  final Ref _ref;
  Future<void> _restore() async {
    final user = await _ref.read(storageProvider).getUser();
    state = AsyncValue.data(user);
  }
  Future<void> login(String email, String password) async {
    state = const AsyncValue.loading();
    try {
      final res = await _ref.read(authRepoProvider).login(LoginRequest(email, password));
      state = AsyncValue.data(res);
    } catch (e, st) {
      state = AsyncValue.error(e, st);
      rethrow;
    }
  }
}
```



```

    }
  }
}

```

Why this shape: this is the exact `AuthController` skeleton from `auth_provider.dart` — loading-first constructor (so a splash/gate screen has something correct to show immediately), a `login()` that sets loading before the call and either data or error after, and re-throwing so the calling screen's own `try/catch` can show a form-level error message too.

21.2 3–4. Home Screen + API Connection

Concept: a screen that, once authenticated, shows a simple dashboard and confirms the token round-trips correctly on a protected endpoint.

Your turn: build a `HomeScreen` (`ConsumerWidget`) that calls one protected `GET` endpoint and displays the result, proving the interceptor is attaching the token.

21.3 5–6. List Data + Details Page

Concept: a `FutureProvider<List<T>>` feeding a `ListView.builder`, each item navigating (with an argument) to a details screen backed by its own `FutureProvider.family` or a simple `get-by-id` call.

Your turn: build `taskListProvider` (`FutureProvider<List<TaskDTO>>`) and `TaskDetailsScreen(taskId: int)` that fetches one task by id.

Solution sketch:

```

final taskListProvider = FutureProvider<List<TaskDTO>>((ref) {
  return ref.watch(taskRepoProvider).getAll();
});

class TaskDetailsScreen extends ConsumerWidget {
  const TaskDetailsScreen({super.key, required this.taskId});
  final int taskId;
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final taskAsync = ref.watch(taskByIdProvider(taskId)); // .family provider
    return Scaffold(
      body: taskAsync.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => Center(child: Text('$e')),
        data: (task) => Text(task.title),
      ),
    );
  }
}

final taskByIdProvider = FutureProvider.family<TaskDTO, int>((ref, id) {
  return ref.watch(taskRepoProvider).getById(id);
});

```

`.family` is new here — it's a provider that takes a parameter (`id`) and caches a *separate* instance of the provider per distinct parameter value. This is the tool you reach for when a `FutureProvider` needs an argument (WorkBridge's job/gig details screens instead pass the `id` through the constructor and call the

repository directly inside `initState`/a local `FutureBuilder`-free approach — `.family` is the more idiomatic Riverpod-native alternative worth knowing).

21.4 7. Search

Concept: the `jobSearchFilterProvider` $\rightarrow$ `jobSearchResultsProvider` pattern from Part 12 — a `StateProvider` holding filter criteria, watched by a `FutureProvider` that re-fetches automatically.

Your turn: replicate it exactly for `TaskDTO`, with a text field that updates `taskFilterProvider.notifier.state` on change.

21.5 8. Form Submission

Concept: a `Form` + `GlobalKey<FormState>` + per-field validators, submitting a typed request DTO on success.

Your turn: build `AddTaskScreen` with title/description fields, validating both are non-empty, posting a `TaskRequestDTO`, and calling `Navigator.pop(true)` on success so the list screen can refresh.

21.6 9–10. Image Upload + File Upload

Concept: `image_picker/file_picker` $\rightarrow$ `File/bytes` $\rightarrow$ `MultipartFile` $\rightarrow$ `FormData` $\rightarrow$ `Dio post`, following the exact `kIsWeb` branching from Part 14.

Your turn: add an optional photo to `AddTaskScreen`, building the `FormData` the same way `saveUserProfile()` does (one JSON part + one optional image part).

21.7 11. Profile

Concept: a `FutureProvider` fetching “my own” resource by the current user’s id (`myUserProfileProvider`’s exact pattern), with 404-as-null handling for “no profile yet.”

21.8 12–13. State Management + Error Handling (tie everything together)

Concept: every screen you built above should already demonstrate this if you followed the pattern — `AsyncValue.when()` for every fetch, `try/catch/finally` with a `_loading` flag for every user-triggered action, and `apiErrorMessage()`-style translation of `DioException` into readable text.

Your turn: write a `String taskBridgeErrorMessage(Object error)` function modeled exactly on `apiErrorMessage()` in `utils/api_client.dart`, and use it in every `catch` block across your mini app instead of showing raw exception text.

21.9 14. Navigation

Concept: a centralized `AppRouter` with named-route constants, an `onGenerateRoute` switch, and an `AuthGate`-style root widget that branches on auth state rather than imperatively navigating.

21.10 15. Logout

Concept: clear stored session, set auth state to `data(null)`, let the gate widget react — no `Navigator.push` needed, exactly as in Part 11.

When you can build all fifteen of these from a blank project without referencing your real WorkBridge code, move to Part 19 — you're ready to read your own codebase as documentation rather than as a black box.

Chapter 22

Part 19 — Reading Your Own Codebase, Guided

Now open your real `work_bridge_flutter` project alongside this section. The goal here isn't new concepts — it's confirming you can *read* the patterns from Parts 1–18 back out of your actual code, unaided.

22.1 ProfileGate — Role-Based Widget Branching

```
class ProfileGate extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final user = ref.watch(currentUserProvider);
    if (user == null) {
      return const Scaffold(body: Center(child: Text('Please sign in to view your profile.')));
    }
    switch (user.role) {
      case UserRole.COMPANY: return const CompanyProfileScreen();
      case UserRole.USER:
      case UserRole.ADMIN:
      case null: return const UserProfileScreen();
    }
  }
}
```

This is the same declarative-gate pattern as `AuthGate` (Part 6), applied one level deeper: instead of gating on “logged in or not,” it gates on *role*, once you already know someone is logged in. `ref.watch(currentUserProvider)` — a `Provider` (not `FutureProvider`) that simply unwraps `authControllerProvider`’s current `.value`, so this widget doesn’t need its own `.when()` here; `AuthGate` above it in the tree already handled the loading/error states, so by the time `ProfileGate` is even reached, a resolved user (or `null`) is guaranteed. Notice the `switch` has to explicitly list every `UserRole` case (`COMPANY`, `USER`, `ADMIN`) plus `null` — Dart’s exhaustiveness checking on enum switches (with `UserRole?` here) means forgetting a case is a compile-time warning, not a runtime surprise; this is a real, quiet safety net worth appreciating rather than an annoyance to work around.

22.2 GigRepository — Confirming the Pattern Holds Across Modules

```
Future<List<GigResponseDTO>> searchGigs(GigSearchRequestDTO request) async {
    final response = await _dio.post(ApiConstants.searchGigs, data: request.toJson());
    return (response.data as List).map((json) => GigResponseDTO.fromJson(json)).toList();
}

Future<GigResponseDTO> getGigById(int id) async {
    final response = await _dio.get(ApiConstants.getGigUrl(id));
    return GigResponseDTO.fromJson(response.data);
}

Future<GigOrderResponseDTO> placeOrder(int gigId, int buyerId) async {
    final response = await _dio.post(ApiConstants.placeGigOrder(gigId, buyerId));
    return GigOrderResponseDTO.fromJson(response.data);
}

Future<GigOrderResponseDTO> acceptQuote(int orderId) async {
    final response = await _dio.patch(ApiConstants.acceptGigQuote(orderId));
    return GigOrderResponseDTO.fromJson(response.data);
}
```

Every one of these is identical in *shape* to `JobRepository`'s methods from Part 8/13 — POST-with-body-search, GET-by-id, POST-to-create, PATCH-to-transition-state — because `GigRepository` is solving the same category of problem (typed CRUD-plus-actions over HTTP) as `JobRepository`, just for a different resource. This is worth noticing explicitly: **once you understand one repository fully, you can read every other repository in the app almost as fast as skimming**, because they all follow the same five-shape vocabulary (get-by-id, search, create, update-field, delete/withdraw). `acceptQuote/rejectQuote` being PATCH rather than PUT is a deliberate signal too — these are narrow state transitions (a gig order moving from “quoted” to “accepted”), not full-resource replacements, matching the same PATCH-for-partial-update reasoning from Part 10's `withdrawApplication` example.

22.3 Cross-Module Consistency Check — Do This Yourself

Pick two files you haven't looked at yet this session — one from `wallet/` and one from `chat/` — and answer these four questions for each, using only what Parts 1–18 taught you:

1. What does the repository's constructor take, and why?
2. Which HTTP verb does each method use, and does it match the “shape vocabulary” above?
3. Does the provider layer above it use `FutureProvider`, `StateNotifierProvider`, or a plain `Provider` — and why that choice, given what the data represents?
4. Where (if anywhere) does the screen layer handle `loading/error/data` explicitly?

If you can answer all four confidently without help, you've internalized the architecture — this is the actual test the instructions ask this book to prepare you for, more so than any single exercise above.

Chapter 23

Part 20 — Designing a New Feature Without AI: “Saved Jobs” Worked Example

This part simulates the exact situation the instructions describe: you need to add a real feature to Work-Bridge, and you’re doing the design yourself. Work through each question *before* reading the “Reasoning” paragraph beneath it — that’s the actual exercise.

The feature: let a signed-in user bookmark (“save”) a job posting from the search results or details screen, see a list of their saved jobs, and unsave one.

23.0.1 Question 1: What backend endpoints does this need, and what HTTP verbs?

Reasoning: Following the shape vocabulary from Part 19 — you need a way to *add* a save (POST /jobs/{jobId}/save or POST /saved-jobs with a body), a way to *remove* one (DELETE /jobs/{jobId}/save or DELETE /saved-jobs/{id}), and a way to *list* the current user’s saved jobs (GET /saved-jobs or GET /users/{userId}/saved-jobs, scoped to the authenticated user via the JWT rather than a client-supplied user id — the same trust boundary your backend already enforces via @PreAuthorize elsewhere). This step lives entirely in your Spring Boot knowledge, not new Flutter knowledge — the point of asking it first is that a Flutter feature can’t be designed sensibly without first knowing the contract it’s built against.

23.0.2 Question 2: What DTOs do you need on the Flutter side?

Reasoning: Following Part 8’s pattern, you’d want a lightweight response DTO — SavedJobResponseDTO with at minimum id, jobId, and probably an embedded JobResponseDTO or just enough summary fields to render a list item without a second round-trip. You likely don’t need a dedicated *request* DTO for saving/unsaving if the endpoint takes the job id as a path parameter (POST /jobs/15/save) rather than a body — not every write operation needs a JSON body; a resource identified purely by its URL path is a completely valid REST design, and building an unnecessary DTO for it would be over-engineering.

23.0.3 Question 3: What does the repository method set look like?

Reasoning: Following GigRepository’s exact shape:

```
class SavedJobRepository {  
    SavedJobRepository(this._apiClient);
```

```

final ApiClient _apiClient;
Dio get _dio => _apiClient.dio;

Future<List<SavedJobResponseDTO>> getSavedJobs() async {
  final response = await _dio.get(ApiConstants.savedJobs);
  return (response.data as List).map((j) => SavedJobResponseDTO.fromJson(j)).toList();
}

Future<void> saveJob(int jobId) async {
  await _dio.post(ApiConstants.saveJobUrl(jobId));
}

Future<void> unsaveJob(int jobId) async {
  await _dio.delete(ApiConstants.saveJobUrl(jobId));
}
}

```

Notice `saveJob/unsaveJob` return `Future<void>` — there’s no meaningful response body to parse for a bare add/remove action, matching the pattern in `withdrawApplication` and `acceptQuote`.

23.0.4 Question 4: What providers do you need, and of which type — walk through it using Part 12’s decision framework?

Reasoning: (a) The list of saved jobs itself is fetch-once, server-owned, async data → `FutureProvider<List<SavedJobResponseDTO>>` (`savedJobsProvider`), same shape as `myUserProfileProvider`. (b) Saving/unsaving needs *behavior* (a method that calls the backend and then needs the list to reflect the change) → this could be a `StateNotifierProvider` wrapping the list with `save()/unsave()` methods that mutate local state after a successful call, **or**, more simply for a project at this stage, just call the repository directly via `ref.read(savedJobRepositoryProvider).saveJob(id)` from the button’s `onPressed`, then `ref.invalidate(savedJobsProvider)` to force a clean refetch — reusing the exact retry mechanism from Part 12’s `jobs_search_Screen.dart` example, rather than introducing a new state-management pattern for a feature this size. **This second option is the more consistent choice given how the rest of WorkBridge is built** — most of your existing mutations (login, applying to a job) go through a controller with real methods, but simpler fire-and-refetch actions don’t need that much machinery. Recognizing “this doesn’t need a new architectural pattern, the existing invalidate-and-refetch approach covers it” is itself a design skill, not a shortcut. (c) “Is job X currently saved” for a list item’s icon → a small derived `Provider` (or a `.family` provider parameterized by job id) that reads `savedJobsProvider`’s current data and checks `.any((s) => s.jobId == jobId)` — purely computed, no new network calls needed.

23.0.5 Question 5: What does the UI layer need to handle, following Part 4/12’s discipline?

Reasoning: A bookmark `IconButton` on each job list item and on the details screen, toggling based on the derived “is saved” provider from 4(c); a dedicated `SavedJobsScreen` with the same `resultsAsync.when/loading/error/data` structure as `JobsSearchScreen`; and — easy to forget — the save/unsave button itself needs a brief local loading/disabled state while its own request is in flight (so a user double-tapping doesn’t fire two overlapping save requests), exactly the same discipline as `LoadingButton` in `widget/common_widget.dart`.

23.0.6 Question 6: What could go wrong, and how does existing WorkBridge code already show you the fix?

Reasoning: A 401 mid-save is already handled globally by the interceptor (Part 11) — no new code needed. A double-tap race is handled by disabling the button while a request is in flight (Question 5). The backend

returning “already saved” (a 409 Conflict, say, if the user taps save twice due to a UI glitch) needs a new `case` in your error-message function — this is exactly the kind of small, deliberate extension `apiErrorMessage()` in `utils/api_client.dart` was designed to accept.

This is the actual exercise the whole book has been building toward: notice that every one of the six answers above reused an existing WorkBridge pattern rather than inventing a new one. That’s not a coincidence — it’s what “understanding the architecture” *means* in practice. The next real feature you build should feel like this: recognizing which existing shape it fits, not starting from a blank page.

Chapter 24

Part 21 — FAQ / Troubleshooting Guide

Q: Why does my `ListView` cause a layout error? A: Almost always the constraint model from Part 4 — a `ListView` wants a bounded height/width from its parent, and a plain `Column/Row` gives its children unbounded space by default. Wrap it in `Expanded` (inside a `Column/Row`) or give it an explicit `SizeBox(height: ...)`.

Q: Why do I get a `NoSuchMethodError` after parsing JSON? A: Almost always a `fromJson` type mismatch (Part 8/16) — the key is missing, misspelled, or the backend sent a different JSON type than your `as String/as int` cast assumed. Print `response.data` raw, before any parsing, to see exactly what arrived, and compare it key-by-key against your DTO.

Q: Why does my provider rebuild too often (or not at all)? A: Too often — you’re likely watching a provider (or a parent widget is) at a wider scope than necessary, so unrelated changes trigger a rebuild of more UI than needed; consider watching a more specific/derived provider instead. Not at all — check you used `ref.watch` (not `ref.read`) inside `build()`, and that the provider you’re watching actually changes when you expect it to (a `StateProvider` needs `.notifier.state = x`, not just mutating a captured variable).

Q: My app works on the emulator but not my browser (or vice versa) — why? A: Almost always Part 15’s host/CORS territory. Check `ApiConstants.host` is resolving to the right value for the platform you’re testing on, and check your browser’s developer console (not just Dio’s exception) for a CORS rejection if you’re on Flutter Web.

Q: `Navigator.pop()` isn’t returning to where I expect / my back button behaves oddly. A: Check whether you meant `push` (adds to the stack) vs `pushReplacement` (swaps the current screen) vs `pushNamedAndRemoveUntil` (clears the stack) — see Part 6’s table. A common bug is using `push` where `pushReplacement` was intended, leaving a “dead” screen underneath that becomes reachable again via back.

Q: I called `setState()` and nothing visibly changed. A: Confirm the field you mutated is actually read inside `build()` — `setState(() { someUnrelatedVariable = x; })` will schedule a rebuild, but if `build()` doesn’t reference the field that actually needs to look different, nothing will visibly change. Also confirm you’re not mutating state on a widget that’s about to be replaced/disposed before the rebuild is processed.

Q: My image doesn’t show up (`Image.network` shows nothing / an error icon). A: Check the URL is actually reachable (paste it into a browser directly); check whether the endpoint requires authentication that `Image.network` can’t provide (Part 14’s authenticated-images gap); check for a `null/empty` filename being concatenated into the URL, producing a URL that resolves to nothing.

Q: `TypeError: type 'Null' is not a subtype of type 'X' on a field I’m sure the backend sends.` A: Log the raw `response.data` for that specific call. It’s very often either (a) the field name doesn’t match

exactly (case-sensitive), (b) the backend only sends that field conditionally (e.g. only for certain roles/states) and you assumed it was always present, or (c) you're parsing the wrong "shape" — e.g. treating a paginated response's wrapper object (`{"content": [...], "totalPages": ...}`) as if it were the raw list itself.

Q: My TextFormField's validator never seems to run. A: Confirm the field is inside a `Form` with a key: `GlobalKey<FormState>()`, and that you're actually calling `_formKey.currentState!.validate()` somewhere (typically at the top of your submit handler) — validators don't run automatically on every keystroke unless you set `autovalidateMode`.

Q: Hot reload "isn't working" after a change. A: If you changed `initState()` logic, a `global/const` value, an enum, or anything outside a `build()` method's reachable code, use hot **restart** instead — hot reload can only inject changed `build()`-reachable code into already-running `State` objects, not re-run initialization that already happened.

Chapter 25

Part 22 — Exercise Index and Self-Assessment

Part	Exercises	Answers located
1 — Dart Fundamentals	7 (3 recall, 3 implementation, 1 problem-solving)	End of Part 2
2 — Async Dart	(folded into Part 1’s answer key — no separate exercises; Part 2 is applied throughout Parts 3+)	—
3 — Flutter Fundamentals	6 (3 recall, 2 implementation, 1 problem-solving)	End of Part 4
4 — Layout	(worked into Part 3’s answers — layout is exercised directly in Part 18’s build tasks)	—
11 — JWT Auth	4 (2 recall, 1 implementation, 1 problem-solving)	End of Part 13
12 — Riverpod	7 (3 recall, 2 implementation, 2 problem-solving)	End of Part 13
14 — File/Image Handling	4 (2 recall, 1 implementation, 1 problem-solving)	End of Part 17
18 — Capstone	15 build tasks across every major concept	Solution sketches inline per task
20 — Feature Design	6 guided design questions (Saved Jobs)	Reasoning inline per question

25.1 Self-Assessment Checklist

Before moving on to independent feature work, you should be able to do each of the following **without opening this book or an AI assistant**:

- ☐ Explain, out loud, why `Widget` is described as “a description, not a thing on screen”
- ☐ Explain why a `StatefulWidget` is really two objects, and why that split exists
- ☐ Write a `fromJson/toJson` pair for a new DTO from a sample JSON response, correctly deciding which fields should be nullable
- ☐ Explain the difference between `ref.watch` and `ref.read`, and correctly choose between them in a new piece of code

- ☐ Explain what `AsyncValue.when()` forces you to handle, and why that's a deliberate constraint, not an inconvenience
- ☐ Trace, from memory, the full path a login request takes from button tap to stored JWT to an authenticated follow-up request
- ☐ Explain why 10.0.2.2 exists and when you need it
- ☐ Correctly decide, for a brand-new feature, whether it needs a `Provider`, `StateProvider`, `FutureProvider`, or `StateNotifierProvider`
- ☐ Explain why `WorkBridge` branches on `kIsWeb` in its file-upload code, and what would break if that branch were removed
- ☐ Build a new screen — Form, validation, loading/error states, network call, navigation on success — without referencing an existing `WorkBridge` screen for the shape of it

If most of these feel solid, Part 18's capstone (if you haven't already done it standalone) is your final check — build `TaskBridge`, then come back and re-attempt any specific item above that still feels shaky.

Chapter 26

Appendix A — Project-Specific Concept Index

Concept	Primary WorkBridge file(s)	Book section
Null safety in DTOs	auth/response/login_response.dart	Part 1
UserRole enum with JSON mapping	auth/request/user_request.dart	Part 1
async/await in a repository call	auth/auth_repository.dart	Part 2
mounted check after await	auth/screen/login_screen.dart	Part 2
StatefulWidget + setState	auth/screen/login_screen.dart	Part 3
loading/error pattern		
dispose() of controllers	auth/screen/login_screen.dart	Part 3, 7
Constraint-model layout	(general — any screen)	Part 4
Named routes + onGenerateRoute	router/app_router.dart	Part 6
Declarative auth gating	auth/screen/auth_gate.dart	Part 6, 11
Role-based widget branching	public_pages/profile_gate.dart	Part 19
Navigator.pop(true) refresh signal	CV sub-entity add/edit screens	Part 6
Form + GlobalKey<FormState> validation	auth/screen/login_screen.dart, register_screen.dart	Part 7
fromJson/toJson with defensive casts	auth/response/login_response.dart	Part 8
List-of-DTOs parsing (cast → map → toList)	job/data/job_repository.dart	Part 8
POST-as-search pattern	job/data/job_repository.dart, gig/data/gig_repository.dart	Part 9
BaseOptions / Dio setup	utils/api_client.dart	Part 10
JWT-attaching interceptor	utils/api_client.dart	Part 10, 11
401 handling / forced logout	utils/api_client.dart	Part 11
NavigationService global navigator key	utils/api_client.dart (via NavigationService)	Part 11
flutter_secure_storage session persistence	services/storage_service.dart	Part 11
Provider dependency chain	auth/providers.dart	Part 12
StateNotifier-based controller	auth/auth_provider.dart	Part 12

Concept	Primary WorkBridge file(s)	Book section
Filter → results reactive FutureProvider chain	job/provider/job_provider.dart	Part 12
404-as-empty-state provider pattern	myUserProfileProvider (cvinformations providers)	Part 12
Repository layering rationale	every *_repository.dart	Part 13
image_picker usage	cvinformations/screens/personal_info_screen.dart	Part 14
file_picker + web/mobile bytes branching	cvinformations/screens/resume_file_screen.dart	Part 14
Multipart upload (FormData, MultipartFile)	cvinformations/data/cv_repository.dart	Part 14
File-serving URL construction	utils/api_constants.dart	Part 14
url_launcher for opening documents	cvinformations/screens/resume_file_screen.dart	Part 14
Platform-aware host resolution	utils/api_constants.dart	Part 15
Error-message translation from DioException	utils/api_client.dart	Part 9, 16

Chapter 27

Appendix B — The 30 Skills You Now Have

A direct answer to “what should this course actually leave me able to do”:

1. Read and write null-safe Dart with confident, deliberate nullability choices
2. Write and reason about `Future/async/await` chains, including error propagation
3. Explain the widget/element/render tree distinction and why it matters for performance
4. Choose correctly between `StatelessWidget` and `StatefulWidget`
5. Diagnose and fix layout overflow/constraint errors from first principles
6. Build a complete, validated form with proper controller lifecycle management
7. Design named routes and a centralized router for a multi-screen app
8. Implement declarative, state-driven navigation gating (auth and role based)
9. Write defensive, correctly-nullable `fromJson/toJson` pairs for any REST DTO
10. Parse nested objects and JSON arrays into typed Dart lists
11. Decide when a REST interaction should be GET/POST/PUT/PATCH/DELETE
12. Configure a Dio instance with `BaseOptions` and sensible timeouts
13. Write a request/response/error interceptor and explain why it belongs there and not per-call
14. Implement JWT storage, attachment, and expiry-triggered logout end-to-end
15. Explain and choose correctly among `Provider`, `StateProvider`, `FutureProvider`, `StateNotifierProvider`
16. Correctly choose `ref.watch` vs `ref.read` vs `ref.listen` in new code
17. Build a reactive filter→results provider chain without manual refetch calls
18. Design a clean UI/controller/repository/API-client layered architecture for a new feature
19. Implement `image_picker` and `file_picker` flows, including web-vs-mobile byte handling
20. Build and send a correct `multipart/form-data` request with mixed JSON + file parts
21. Construct and reason about file-serving URLs distinct from JSON API URLs
22. Diagnose and resolve the `localhost` vs `10.0.2.2` platform confusion
23. Reason about CORS as a browser-only constraint specific to Flutter Web
24. Follow a systematic error-diagnosis process instead of guessing at fixes
25. Write basic unit tests for DTO parsing and repository logic using mocked dependencies
26. Override Riverpod providers for isolated, dependency-free testing
27. Read an unfamiliar repository/provider pair in an existing large codebase quickly, by pattern-matching to known shapes
28. Design a new full-stack-adjacent feature (endpoints → DTOs → repository → providers → UI) from a plain-English description
29. Recognize when an existing architectural pattern already fits a new problem, rather than inventing a new one unnecessarily
30. Build a complete, working small Flutter app — auth, list, detail, form, file upload, search, logout — entirely from a blank project

Chapter 28

Closing Note

Everything in this book was pulled from your actual `work_bridge_flutter` repository, not a generic tutorial retrofitted with your project's name. Where the code was excellent, this book said so and explained why. Where it had a rough edge — the 2000-second timeout, the unchecked `settings.arguments as int` cast, the `Image.network` authentication gap — this book named it plainly rather than smoothing it over, because knowing *why* something is a tradeoff (and not a mistake to be ashamed of) is exactly the judgment you're trying to build.

The real test isn't finishing this book. It's whether, the next time you need to add a feature to WorkBridge, you reach for Part 20's reasoning process instead of an AI assistant — and if a fresh Flutter concept comes up mid-feature that this book didn't cover, whether you can now teach it to yourself the same way this book taught you the ones it did cover.